Universal Simulator

Documentation

Version 3.0.34

Modules and classes

1 Module: boxes 10
1.1 Class: Accumulator 10
1.1.1 Inputs 10
1.1.2 Outputs 10
1.1.3 Usage 10
1.2 Class: AhFromRh 11
1.2.1 Inputs 11
1.2.2 Outputs 11
1.2.3 Usage 11
1.3 Class: AllTypes 11
1.3.1 Inputs 11
1.3.2 Usage 12
1.4 Class: Briere 12
1.4.1 Inputs 12
1.4.2 Outputs 12
1.4.3 Background 12
1.4.4 Parameter estimation 13
1.4.5 Usage 14
1.5 Class: Buffer 15
1.5.1 Inputs 15
1.5.2 Outputs 15
1.5.3 Usage 16
1.6 Class: Calendar 16
1.6.1 Inputs 16
1.6.2 Outputs 16
1.6.3 Usage 16
1.7 Class: Combinations 16
1.7.1 Outputs 16
1.7.2 Usage 17
1.8 Class: Computation 17
1.8.1 Outputs 17
1.8.2 Usage 17
1.9 Class: Date 17
1.9.1 Inputs 17
1.9.2 Outputs 17
1.9.3 Usage 17
1.10 Class: DateSplit 17
1.10.1 Inputs 17
1.10.2 Outputs 17
1.10.3 Usage 17
1.11 Class: DateTime 18
1.11.1 Inputs 18
1.11.2 Outputs 18
1.11.3 Usage 18
1.12 Class: DateTimeSignal 18
1.12.1 Inputs 18
1.12.2 Outputs 18
1.12.3 Usage 19
1.12.4 Circadian signals 19
1.12.5 Non-circadian signals 21
1.13 Class: DayDegrees 22

1.13.1 Inputs 22
1.13.2 Outputs 23
1.13.3 Usage 23
1.14 Class: DemandBudget 23
1.14.1 Inputs 23
1.14.2 Outputs 23
1.14.3 Usage 23
1.15 Class: DewPoint 23
1.15.1 Inputs 23
1.15.2 Outputs 23
1.15.3 Usage 23
1.16 Class: Distribution 23
1.16.1 Inputs 23
1.16.2 Usage 24
1.17 Class: FixedSignal 24
1.17.1 Inputs 24
1.17.2 Outputs 24
1.17.3 Usage 24
1.18 Class: FoodWeb 24
1.18.1 Inputs 24
1.18.2 Usage 24
1.19 Class: FoodWebCalcParasiteHost 24
1.19.1 Inputs 24
1.19.2 Usage 24
1.20 Class: FoodWebCalcPredatorPrey 25
1.20.1 Inputs 25
1.20.2 Usage 25
1.21 Class: FunctionalResponse 25
1.21.1 Inputs 25
1.21.2 Outputs 25
1.22 Class: Host 25
1.22.1 Inputs 25
1.22.2 Outputs 25
1.22.3 Usage 26
1.23 Class: Hump 26
1.23.1 Inputs 26
1.23.2 Outputs 26
1.23.3 Usage 26
1.24 Class: IgnoredBox 28
1.24.1 Usage 28
1.25 Class: LobryRossoFlandrois 28
1.25.1 Inputs 28
1.25.2 Outputs 28
1.25.3 Usage 28
1.26 Class: Maker 29
1.26.1 Inputs 29
1.26.2 Usage 29
1.27 Class: Maximum 33
1.27.1 Inputs 33
1.27.2 Outputs 33
1.28 Class: MaximumAt 33
1.28.1 Inputs 33
1.28.2 Outputs 33
1.28.3 Background 33

1.28.4 Usage 33
1.29 Class: Mean 35
1.29.1 Inputs 35
1.29.2 Outputs 35
1.30 Class: Message 36
1.30.1 Inputs 36
1.31 Class: Minimum 36
1.31.1 Inputs 36
1.31.2 Outputs 36
1.32 Class: Multiplum 36
1.32.1 Inputs 36
1.32.2 Outputs 36
1.33 Class: NumberedFile 36
1.33.1 Inputs 36
1.33.2 Outputs 36
1.34 Class: OffsetDateTime 37
1.34.1 Inputs 37
1.34.2 Outputs 37
1.35 Class: OnOff 37
1.35.1 Inputs 37
1.35.2 Outputs 37
1.35.3 Usage 37
1.36 Class: OnOffByDate 37
1.36.1 Inputs 37
1.36.2 Outputs 38
1.36.3 Usage 38
1.37 Class: OutputR 38
1.37.1 Inputs 38
1.37.2 Outputs 38
1.38 Class: OutputSelector 39
1.38.1 Inputs 39
1.38.2 Outputs 39
1.38.3 Usage 39
1.38.4 Skip a number of lines 39
1.38.5 Skip lines until a certain time 40
1.38.6 Periodic output 40
1.38.7 Final output 41
1.39 Class: OutputText 41
1.39.1 Inputs 41
1.40 Class: OutputWriter 42
1.40.1 Inputs 42
1.40.2 Outputs 42
1.40.3 Usage 42
1.40.4 Output folder 42
1.40.5 Output file format 43
1.41 Class: PageR 43
1.41.1 Inputs 43
1.42 Class: Parasite 43
1.42.1 Inputs 43
1.42.2 Outputs 44
1.42.3 Usage 44
1.43 Class: PlotR 44
1.43.1 Inputs 44
1.44 Class: PopUp 44

1.44.1 Inputs 44
1.44.2 Outputs 45
1.45 Class: Population 45
1.45.1 Inputs 45
1.45.2 Outputs 45
1.45.3 Usage 45
1.46 Class: Predator 45
1.46.1 Inputs 45
1.46.2 Outputs 46
1.46.3 Usage 46
1.47 Class: Prey 46
1.47.1 Inputs 46
1.47.2 Outputs 46
1.47.3 Usage 46
1.48 Class: PrioritySignal 46
1.48.1 Inputs 46
1.48.2 Outputs 46
1.49 Class: ProportionalSignal 47
1.49.1 Inputs 47
1.49.2 Outputs 47
1.50 Class: RandomBinomial 47
1.50.1 Inputs 47
1.50.2 Outputs 47
1.50.3 Usage 48
1.51 Class: RandomLogNormal 48
1.51.1 Inputs 48
1.51.2 Outputs 49
1.52 Class: RandomLogUniform 49
1.52.1 Inputs 49
1.52.2 Outputs 49
1.53 Class: RandomNormal 49
1.53.1 Inputs 49
1.53.2 Outputs 49
1.54 Class: RandomUniform 50
1.54.1 Inputs 50
1.54.2 Outputs 50
1.55 Class: RandomUniformInt 50
1.55.1 Inputs 50
1.55.2 Outputs 50
1.56 Class: RandomiserMonteCarlo 50
1.56.1 Inputs 50
1.56.2 Outputs 51
1.57 Class: RandomiserSobolSequence 51
1.57.1 Inputs 51
1.57.2 Outputs 51
1.57.3 Usage 51
1.58 Class: RandomiserStratified 54
1.58.1 Inputs 54
1.58.2 Outputs 54
1.59 Class: Ratio 54
1.59.1 Inputs 54
1.59.2 Outputs 54
1.60 Class: Records 55
1.60.1 Inputs 55

1.60.2 Outputs 55
1.60.3 Daily readings 55
1.60.4 Interpolation 56
1.60.5 Hourly readings 57
1.60.6 Ignoring the year 58
1.60.7 Extrapolation 59
1.60.8 Recycling the year 59
1.61 Class: RunningAverageSteps 60
1.61.1 Inputs 60
1.61.2 Outputs 60
1.61.3 Usage 60
1.62 Class: RunningAverageTimed 60
1.62.1 Inputs 60
1.62.2 Outputs 60
1.62.3 Usage 60
1.63 Class: RunningSumSteps 61
1.63.1 Inputs 61
1.63.2 Outputs 61
1.63.3 Usage 61
1.64 Class: RunningSumTimed 61
1.64.1 Inputs 61
1.64.2 Outputs 61
1.64.3 Usage 61
1.65 Class: Sah 61
1.65.1 Inputs 61
1.65.2 Usage 61
1.66 Class: Sandy 62
1.66.1 Inputs 62
1.66.2 Outputs 62
1.66.3 Usage 62
1.67 Class: Scenarios 62
1.67.1 Inputs 62
1.67.2 Outputs 62
1.67.3 Usage 62
1.68 Class: SelectFile 62
1.68.1 Inputs 62
1.68.2 Outputs 63
1.68.3 Usage 63
1.69 Class: SequenceByIteration 63
1.69.1 Inputs 63
1.69.2 Outputs 63
1.69.3 Usage 63
1.70 Class: SequenceByStep 63
1.70.1 Inputs 63
1.70.2 Outputs 63
1.70.3 Usage 64
1.71 Class: SeriesByIteration 64
1.71.1 Inputs 64
1.71.2 Outputs 64
1.71.3 Usage 64
1.72 Class: SeriesByStep 64
1.72.1 Inputs 64
1.72.2 Outputs 64
1.72.3 Usage 64

1.73 Class: Simulation 64
1.73.1 Inputs 64
1.73.2 Outputs 65
1.73.3 Usage 65
1.74 Class: SineWaveTemperature 65
1.74.1 Inputs 65
1.74.2 Outputs 65
1.74.3 Usage 66
1.75 Class: SobolGFunction 66
1.75.1 Inputs 66
1.75.2 Outputs 66
1.75.3 Usage 66
1.76 Class: Split 66
1.76.1 Inputs 66
1.76.2 Outputs 66
1.76.3 Usage 66
1.77 Class: Stage 66
1.77.1 Inputs 66
1.77.2 Outputs 67
1.77.3 Usage 67
1.78 Class: StageAndPhase 67
1.78.1 Inputs 67
1.78.2 Outputs 68
1.78.3 Usage 68
1.79 Class: Sum 68
1.79.1 Inputs 68
1.79.2 Outputs 68
1.79.3 Usage 68
1.80 Class: Sun 69
1.80.1 Inputs 69
1.80.2 Outputs 69
1.80.3 Usage 69
1.81 Class: SunDiffuseRadiation 69
1.81.1 Inputs 69
1.81.2 Outputs 69
1.81.3 Usage 69
1.82 Class: SuperFunctionalResponse 70
1.82.1 Inputs 70
1.82.2 Outputs 70
1.82.3 Usage 70
1.83 Class: SupplyBudget 70
1.83.1 Inputs 70
1.83.2 Outputs 70
1.83.3 Usage 70
1.84 Class: ThresholdSignal 71
1.84.1 Inputs 71
1.84.2 Outputs 71
1.84.3 Usage 71
1.85 Class: TimeToHours 71
1.85.1 Inputs 71
1.85.2 Outputs 71
1.85.3 Usage 71
1.86 Class: VectorScaled 71
1.86.1 Inputs 71

1.86.2 Outputs 72
1.86.3 Usage 72
1.87 Class: VectorSum 72
1.87.1 Inputs 72
1.87.2 Outputs 72
1.87.3 Usage 72
1.88 Class: boxesDocumentation 72
1.88.1 Inputs 72
1.88.2 Usage 72
2 Module: callosobruchus 73
2.1 Class: DensityDependent 73
2.1.1 Inputs 73
2.1.2 Outputs 73
2.1.3 Usage 73
2.2 Class: Fecundity 73
2.2.1 Inputs 73
2.2.2 Outputs 73
2.2.3 Usage 73
2.3 Class: Stock 73
2.3.1 Inputs 73
2.3.2 Outputs 74
2.3.3 Usage 74
2.4 Class: callosobruchusDocumentation 74
2.4.1 Inputs 74
2.4.2 Usage 74
3 Module: cmbp 75
3.1 Class: DevRate 75
3.1.1 Inputs 75
3.1.2 Outputs 75
3.1.3 Usage 75
3.2 Class: Emigration 75
3.2.1 Inputs 75
3.2.2 Outputs 75
3.2.3 Usage 75
3.3 Class: ExponentialPeak 75
3.3.1 Inputs 75
3.3.2 Outputs 75
3.3.3 Usage 76
3.4 Class: Fecundity 76
3.4.1 Inputs 76
3.4.2 Outputs 76
3.4.3 Usage 76
3.5 Class: GrainMoistureContent 76
3.5.1 Inputs 76
3.5.2 Outputs 76
3.5.3 Usage 76
3.6 Class: GrainStore 76
3.6.1 Inputs 76
3.6.2 Outputs 77
3.6.3 Usage 77
3.7 Class: Immigration 77
3.7.1 Inputs 77
3.7.2 Outputs 77
3.7.3 Usage 77

3.8 Class: SurvivalByDensity 77
3.8.1 Inputs 77
3.8.2 Outputs 77
3.8.3 Usage 77
4 Module: command 78
4.1 Class: commandDocumentation 78
4.1.1 Inputs 78
4.1.2 Usage 78

1 Module: boxes

1.1 Class: Accumulator

1.1.1 Inputs

Parameter	Type	Default	Description
`initial`	`double`	`0.0`	Initial `value` at reset
`change`	`double`	`0.0`	Change added to `value`
`minValue`	`double`	$-\infty$	Minimum possible `value`
`maxValue`	`double`	∞	Maximum possible `value`

1.1.2 Outputs

Parameter	Type	Default	Description
`value`	`double`	`0.0`	Accumulated value
`signal`	`double`	`0.0`	Synonym for `value`
`isAtMin`	`bool`	`FALSE`	Is `value` equal to `minValue`?
`isAtMax`	`bool`	`FALSE`	Is `value` equal to `maxValue`?

1.1.3 Usage

The `Accumulator` box keeps track of an accumulated sum. For every simulation step, change (can be negative) is added to its `value`. This boxscript calculates the day-degrees above a threshold of 10 °C (a DayDegrees box would allow you more control with less BoxScript code):

```
// accumulator1.box
Simulation sim {
  Calendar calendar {
    .begin = 01/05/2009
    .end   = 30/09/2009
  }
  Records weather {
    .fileName = "weather/flakkebjerg 2009.txt"
  }
  Accumulator dd18 {
    .change = if (weather[Tavg] > 18) then weather[Tavg] - 18 else 0
  }
  OutputR {
    PageR {
      .xAxis = calendar[date]
      PlotR {
        .layout = "merged"
        .ports = weather[Tavg] | dd18[change] | dd18[value]
      }
    }
  }
}
```

Note that in the output, if a port is named “value” then it is by default labelled after its parent name (here “dd18”):

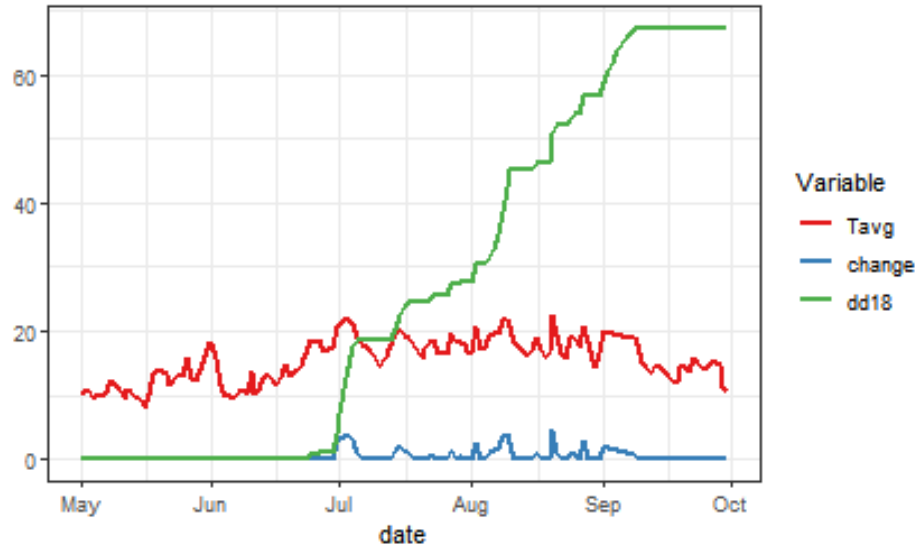


Figure 1: Accumulator example

1.2 Class: AhFromRh

1.2.1 Inputs

Parameter	Type	Default	Description
`temperature`	`double`	`0.0` °C	Temperature
`rh`	`double`	`0.0 [0;100]`	Relative humidity

1.2.2 Outputs

Parameter	Type	Default	Description
`value`	`double`	`0.0 kg/m3`	Absolute humidity

1.2.3 Usage

`AhFromRh` computes absolute humidity (kg/m^3) from relative humidity (%) and temperature (oC).

1.3 Class: AllTypes

1.3.1 Inputs

Parameter	Type	Default	Description
`bool`	`bool`	`FALSE`	
`int`	`int`	`0`	
`double`	`double`	`0.0`	
`string`	`string`		
`date`	`date`	`null`	
`time`	`time`	`null`	
`datetime`	`datetime`	`null`	
`baredate`	`baredate`	`null`	
`path`	`path`	`null`	
`vec_bool`	`vec_bool`	`null`	
`vec_int`	`vec_int`	`null`	
`vec_double`	`vec_double`	`null`	

Parameter	Type	Default	Description
vec_string	vec_string	null	
vec_date	vec_date	null	
vec_time	vec_time	null	
vec_datetime	vec_datetime	null	
vec_baredate	vec_baredate	null	

1.3.2 Usage

`AllTypes` contains one port of each type - useful to learn about types.

1.4 Class: Briere

1.4.1 Inputs

Parameter	Type	Default	Description
T	double	a) °C	
timeStepDays	double	b) d	
resetTotal	bool	FALSE	Reset `total` zero?
isTicking	bool	TRUE	Is the physiological time running? Else `step` will be zero
T0	double	0.0 °C	Lower temperature threshold
Tmax	double	0.0 °C	Upper temperature threshold
a	double	0.0 /°C	Scaling parameter

a) `weather[Tavg]`
b) `calendar[timeStepDays]`

1.4.2 Outputs

Parameter	Type	Default	Description
step	double	0.0 PTU	Increment in physiological time units (PTU)
total	double	0.0 PTU	Total time since reset

1.4.3 Background

The `Briere` box implements the non-linear model for physiological development of Briere et al. (1999) defined by three inputs: `T0`, `Tmax` and `a`. They designate the lower and upper threshold for development, respectively, and a scaling parameter with no direct biological meaning. The equation for development rate is

$$\frac{1}{L} = aT(T - T_0)\sqrt{T_{\max} - T}$$

The position of the temperature optimum is given implicitly by T_0 and $T_{\max}$ and can be derived from the above equation as

$$T_{\text{opt}} = \frac{\sqrt{9T_0^2 - 16T_0T_{\max} + 16T_{\max}^2} + 3T_0 + 4T_{\max}}{10}$$

Maybe of more relevance is the finding, that if you want to define the curve by its optimum rather than its maximum temperature, you can calculate the maximum temperature from the optimal temperature by

$$T_{\max} = \frac{T_{\text{opt}}\left(5T_{\text{opt}} - 3T_0\right)}{4T_{\text{opt}} - 2T_0}$$

1.4.4 Parameter estimation

The three parameters of the curve can be estimated by non-linear regression. Here is a typical data set for temperature-dependent insect development rate. It shows the development time (L) and rate ($1/L$) at different temperatures (T) for brown marmorated stink bug (*Halyomorpha halys*) (from Nielsen et al. (2008)):

T [°C]	L [d]	$\frac{1}{L}$ [d^{-1}]
15	∞	0.0000
17	121.50	0.0082
20	81.07	0.0123
25	44.65	0.0224
27	36.67	0.0273
30	33.73	0.0296
33	37.67	0.0265
35	∞	0.0000

Using R *nls* we arrive at the estimates, $T_0 = 14.2$ °C, $T_{\max} = 35.0$ °C and $a = 2.836 \times 10^{-5}$ d^{-1} that we can plug into this boxscript, which also shows two alternative models (simple linear and the DayDegrees box, which features a breakpoint at the optimum temperature):

```
// briere1.box
Simulation sim {
  .steps = temperature[steps]
  SequenceByStep temperature {
    .min = 12
    .max = 37
    .by = 0.2
  }
  DayDegrees dayDegrees {
    .timeStepDays = 1
    .T0 = 14.2
    .Topt = 29
    .Tmax = 35
    .T = temperature[value]
  }
  Briere briere {
    .timeStepDays = 1
    .T0 = 14.2
    .Tmax = 35
    .a = 2.836e-5
    .T = temperature[value]
  }
  OutputR {
    .scripts = "briere1.R"
    OutputText {
      .ports = temperature[value] | dayDegrees[step] | briere[step]
    }
  }
}
```

This is the plot produced:

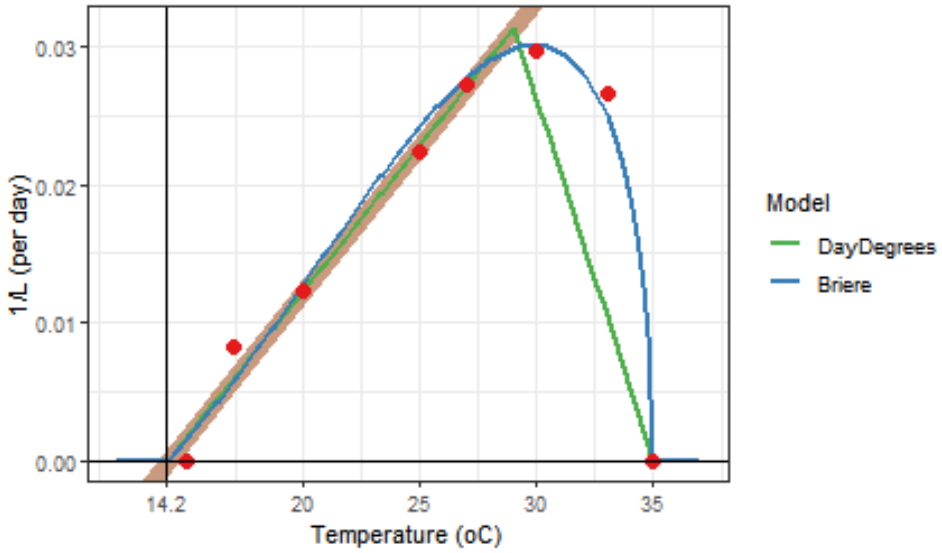


Figure 2: Briere development-rate example

1.4.5 Usage

In practical usage, you often want to turn the ticking of the physiological clock on and off. Sometimes, you may want to reset it to zero too. This is what the two remaining input ports (`isTicking` and `resetTotal`) are for. They are both of the boolean type. In this script, the clock is only ticking in the period 1 May to 1 October. Moreover, it is reset to zero on 1 May and 1 July:

```
// briere2.box
Simulation sim {
  Calendar calendar {
    .begin = 01/01/2009
    .end   = 31/12/2010
  }
  Records weather {
    .fileName = "weather/flakkebjerg 2009.txt"
    .cycle = TRUE
  }
  Briere clock {
    .isTicking = (calendar[date] >= 01/05) && (calendar[date] < 01/10)
    .resetTotal = (calendar[date] == 01/05) || (calendar[date] == 01/07)
    .T0 = 8
    .Tmax = 32
    .a = 2.836e-5
  }
  OutputR {
    PageR {
      .xAxis = calendar[date]
      PlotR {
        .ports = clock[output::*] | clock[isTicking] | clock[resetTotal]
        .ggplot = "scale_x_datetime(breaks=date_breaks('3 months'),
                                    labels = date_format ('%b')) +
                   labs(x='2009 - 2010')"
      }
    }
  }
}
```

Noticable BoxScript tricks include

- the use of the `cycle` port in `Records` to recycle the weather data;
- the use of *bare dates* to ignore the year of the date (e.g., `01/05` to represent 1 May any year);
- the use of the `ggplot` port to `PlotR` to add details to the plot produced.

The output shows two years worth of data, identical since the weather of 2009 is used for both years:

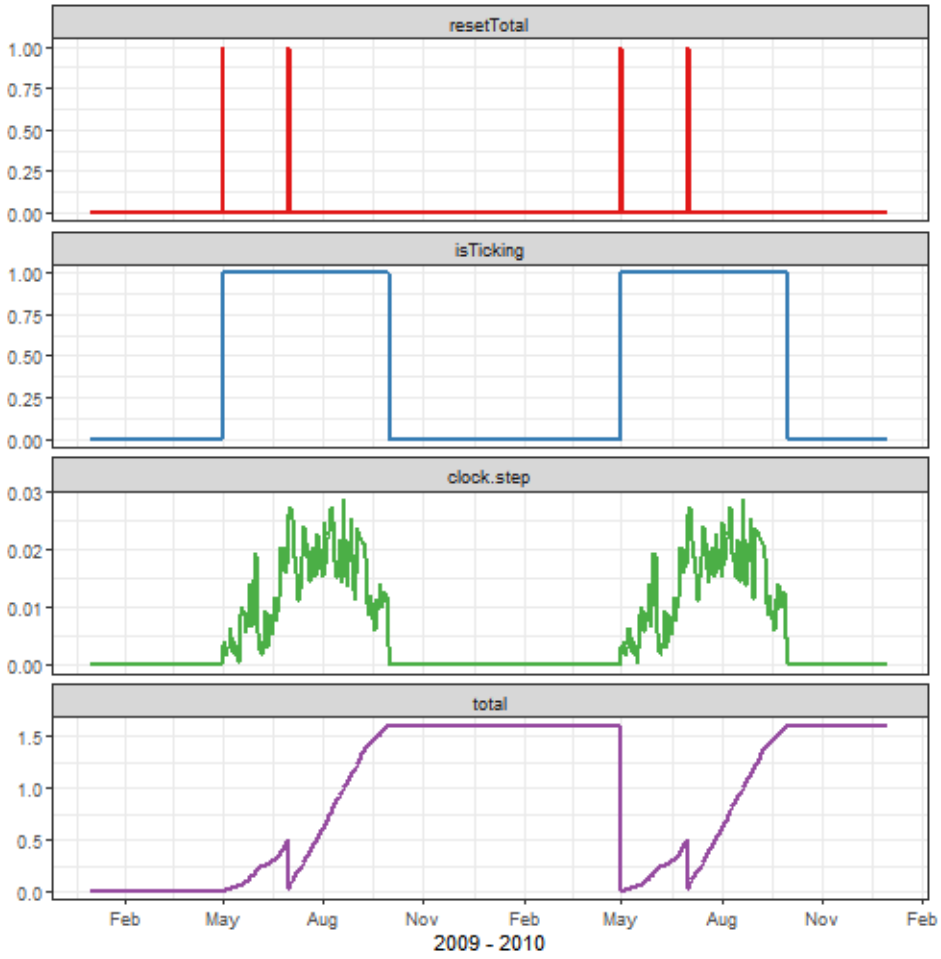


Figure 3: Briere clock switched on, off and reset

1.5 Class: Buffer

1.5.1 Inputs

Parameter	Type	Default	Description
`initial`	`double`	`0.0`	Initial amount at first place in buffer
`input`	`double`	`0.0`	Input to next place in buffer
`increment`	`double`	`0.0`	Increment added to every place in buffer
`size`	`int`	`100`	Number of places in buffer

1.5.2 Outputs

Parameter	Type	Default	Description
`sum`	`double`	`0.0`	Total amount in buffer
`average`	`double`	`0.0`	Average amount of places used in buffer
`buffer`	`vec_double`	`null`	The buffer itself

1.5.3 Usage

`Buffer` is a first in-first out buffer.

1.6 Class: Calendar

1.6.1 Inputs

Parameter	Type	Default	Description
`latitude`	`double`	`52.0` `[-90,90]`	Latitude
`longitude`	`double`	`11.0` `[-180,180]`	Longitiude
`timeZone`	`int`	`1 h`	Time zone
`begin`	`datetime`	a)	When simulation begins
`end`	`datetime`	`null`	When simulation ends (optional)
`timeStep`	`int`	`1` `>0`	Time step in units of timeUnit
`timeUnit`	`string`	`"d"`	Unit of time step (y|d|h|m|s)

a) `2000/01/01T00:00:00`

1.6.2 Outputs

Parameter	Type	Default	Description
`steps`	`int`	`0`	Number of steps from begin to end
`stepWithinDay`	`int`	`0`	Step number within day
`stepsPerDay`	`int`	`0`	Number of steps per day
`date`	`date`	`null`	Current date
`time`	`time`	`null`	Current time of the day
`dateTime`	`datetime`	`null`	Current date and time
`atMidnight`	`bool`	`FALSE`	Just past midnight?
`timeStepSecs`	`double`	`0.0 s`	Time step duration
`timeStepDays`	`double`	`0.0 d`	Time step duration
`totalTimeSteps`	`int`	`0`	Total number of time steps since calendar was reset
`totalTime`	`int`	`0`	Total time since calendar was reset; in units of `timeUnit`
`totalDays`	`double`	`0.0 d`	Total time since calendar was reset in days
`dayOfYear`	`int`	`0` `[1;366]`	Julian day
`bareDate`	`baredate`	`null`	`date` as a bare date

1.6.3 Usage

`Calendar` keeps check on date and time.

1.7 Class: Combinations

1.7.1 Outputs

Parameter	Type	Default	Description
`iterations`	`int`	`0`	Total number of combinations

1.7.2 Usage

`Combinations` produces all combinations of child values.

1.8 Class: Computation

1.8.1 Outputs

Parameter	Type	Default	Description
`step`	`string`	`construct`	Name of current computation step

1.8.2 Usage

`Computation` provides information on the computation schedule.

1.9 Class: Date

1.9.1 Inputs

Parameter	Type	Default	Description
`dmy`	`string`	`DMY`	A date string
`day`	`int`	`1 [1;31]`	Day of month
`month`	`int`	`1 [1;12]`	Month
`year`	`int`	`2000`	Year

1.9.2 Outputs

Parameter	Type	Default	Description
`date`	`date`	`null`	The date constructed
`value`	`date`	`null`	Synonym for `date`

1.9.3 Usage

`Date` constructs a date from a DMY string, or from day, month and year.

1.10 Class: DateSplit

1.10.1 Inputs

Parameter	Type	Default	Description
`date`	`date`	`null`	The date to split

1.10.2 Outputs

Parameter	Type	Default	Description
`day`	`int`	`0`	Day of month
`month`	`int`	`0`	Month
`year`	`int`	`0`	Year

1.10.3 Usage

`DateSplit` splits a date into day, month and year.

1.11 Class: DateTime

1.11.1 Inputs

Parameter	Type	Default	Description
dateTimeString	string	datetime	A datetime string
day	int	1 [1;31]	Day of month
month	int	1 [1;12]	Month
year	int	2000	Year
hour	int	0 [0:24]	Hour of the day
minute	int	0 [0:59]	Minute of the hour
second	int	0 [0:59]	Second of the minute

1.11.2 Outputs

Parameter	Type	Default	Description
dateTime	datetime	null	The datetime constructed
value	datetime	null	Synonym for dateTime

1.11.3 Usage

DateTime constructs a datetime from a DMYHMS string, or from day, month, year, hour, minute and second.

1.12 Class: DateTimeSignal

1.12.1 Inputs

Parameter	Type	Default	Description
initialSignal	double	0.0	Value of signal at reset
initialFlag	bool	FALSE	Value of flag at reset
currentDateTime	datetime	a)	
beginDate	baredate	1/1	The beginning of the date interval
endDate	baredate	31/12	The end of the date interval
beginTime	time	00:00:00	The beginning of the time interval
endTime	time	00:00:00	The end of the time interval
signalInside	double	1.0	Resulting signal when day and time are inside the day and time intervals
signalOutside	double	0.0	Resulting signal when day and time are outside the day and time intervals
circadian	bool	TRUE	Tells whether signal follows a daily rythm

a) calendar[dateTime]

1.12.2 Outputs

Parameter	Type	Default	Description
signal	double	0.0	Value of the signal
flagIsUp	bool	FALSE	Is the signal on?
flagJustRaised	bool	FALSE	Was the signal just set on?
flagJustLowered	bool	FALSE	Was the signal just set off?
value	double	0.0	Synonym for signal

1.12.3 Usage

The `DateTimeSignal` box sets the `signal` to `signalInside` when the `currentDateTime` is inside the set limits, otherwise it is set to `signalOutside`. The limits are set by date (`beginDate` and `endDate`) and time of the day (`beginTime` and `endTime`) separately. Note that `beginDate` and `endDate` are *bare dates*, which means that they just specify the day and month, not the year. This means that you can have a calendar interval across 1 January, e.g. from 1 December (`beginDate`) to 1 March (`endDate`). All intervals of dates and times are closed (i.e. limits are inclusive). If `beginTime` and `endTime` keep their default value (`00:00`), the time of day is ignored.

If `circadian` is `TRUE` then `currentDateTime` is considered inside the limits if the date of `currentDateTime` is inside the [`beginDate`; `endDate`] period, and the time of `currentDateTime` is inside the [`beginTime`; `endTime`] period (specifying a recurring signal).

If `circadian` is `FALSE` then `currentDateTime` is considered inside the limits if it falls inside the period from `beginTime` on `beginDate` to `endTime` on `endDate` (specifying a single signal).

1.12.4 Circadian signals

The difference that `circadian` makes is easier to understand by example. First we demonstrate a circadian signal:

```
// date_time_signal1.box
Simulation sim {
  Calendar calendar {
    .begin = 01/01/2001T12:00
    .end   = 10/01/2001T12:00
    .timeStep = 15
    .timeUnit = "m"
  }
  DateTimeSignal signal {
    .beginDate  = 03/01/2001
    .endDate    = 07/01/2001
    .beginTime  = 14:00
    .endTime    = 20:00
    .initialSignal = 50
    .signalInside  = 80
    .signalOutside = 50
  }
  OutputR {
    PageR {
      .xAxis = calendar[dateTime]
      PlotR {
        .ports = signal[value] | signal[flagIsUp] |
                 signal[flagJustRaised] | signal[flagJustLowered]
        .ggplot = "scale_x_datetime(breaks=date_breaks('days'),
                                    labels = date_format ('%d')) +
                   labs(x='January')"
      }
    }
  }
}
```

The boxscript raises the signal for five days between 14:00 and 20:00 every day, as shown in the output:

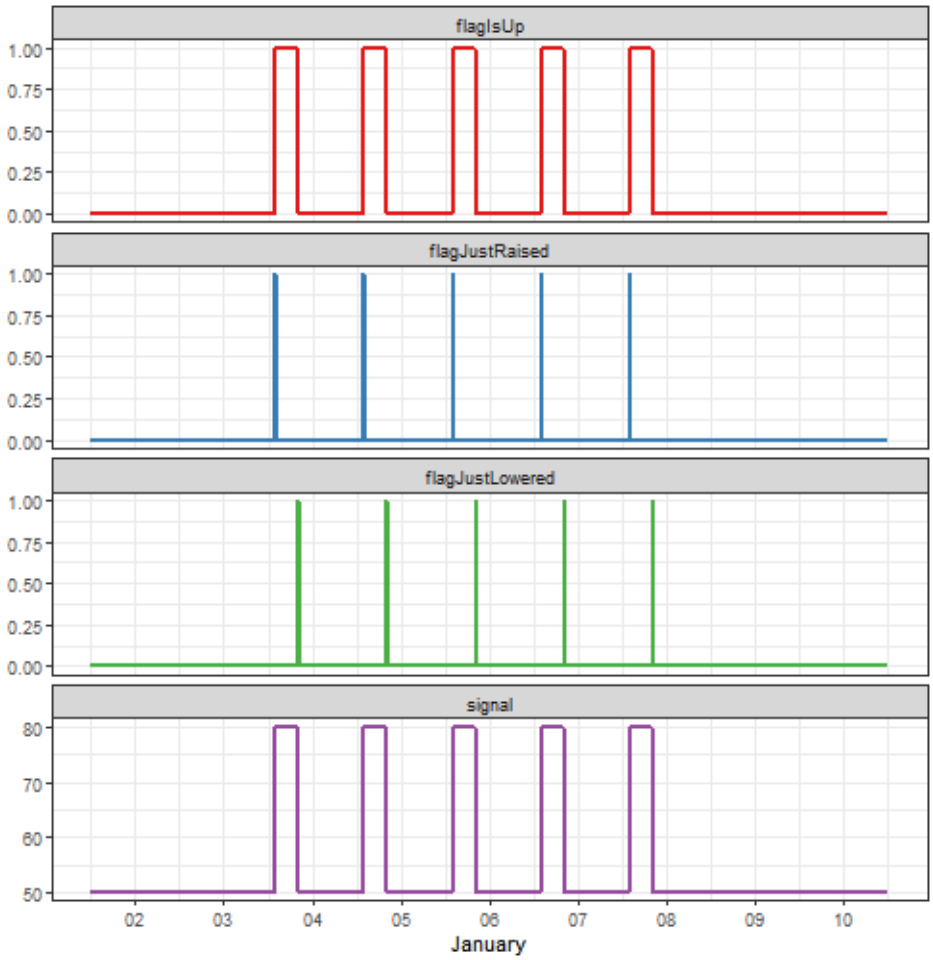


Figure 4: Circadian DateTimeSignal

You can check the precision of the signal going *on* at the Universal Simulator prompt with the `rows 295 300` command:

```
> rows 295 300
...
           dateTime flagIsUp flagJustRaised flagJustLowered signal iteration step
2001/01/04T13:30:00        0              0               0   50.0         1  294
2001/01/04T13:45:00        0              0               0   50.0         1  295
2001/01/04T14:00:00        1              1               0   80.0         1  296
2001/01/04T14:15:00        1              0               0   80.0         1  297
2001/01/04T14:30:00        1              0               0   80.0         1  298
2001/01/04T14:45:00        1              0               0   80.0         1  299
```

Or at the R prompt with `sim[295:300,]`:

```
> sim[295:300,]
              dateTime flagIsUp flagJustRaised flagJustLowered signal iteration step
295 2001-01-04 13:30:00        0              0               0     50         1  294
296 2001-01-04 13:45:00        0              0               0     50         1  295
297 2001-01-04 14:00:00        1              1               0     80         1  296
298 2001-01-04 14:15:00        1              0               0     80         1  297
299 2001-01-04 14:30:00        1              0               0     80         1  298
300 2001-01-04 14:45:00        1              0               0     80         1  299
```

Similarly, you can check the timeliness of the signal going *off*:

```
> rows 320 325
...
           dateTime flagIsUp flagJustRaised flagJustLowered signal iteration step
```

```
2001/01/04T19:45:00       1             0             0   80.0        1  319
2001/01/04T20:00:00       1             0             0   80.0        1  320
2001/01/04T20:15:00       0             0             1   50.0        1  321
2001/01/04T20:30:00       0             0             0   50.0        1  322
2001/01/04T20:45:00       0             0             0   50.0        1  323
2001/01/04T21:00:00       0             0             0   50.0        1  324
```

1.12.5 Non-circadian signals

In the next script (date_time_signal2.box), we only changed one detail, namely the value of `circadian` in the `signal` box:

```
DateTimeSignal signal {
  .beginDate  = 03/01/2001
  .beginTime  = 14:00
  .endDate    = 07/01/2001
  .endTime    = 20:00
  .initialSignal = 50
  .signalInside  = 80
  .signalOutside = 50
  .circadian = FALSE
}
```

We also re-arranged the ordering of the date and time inputs to make their meaning more clear. The output shows that the signal is raised only through one time period stretching 4 days and 6 hours:

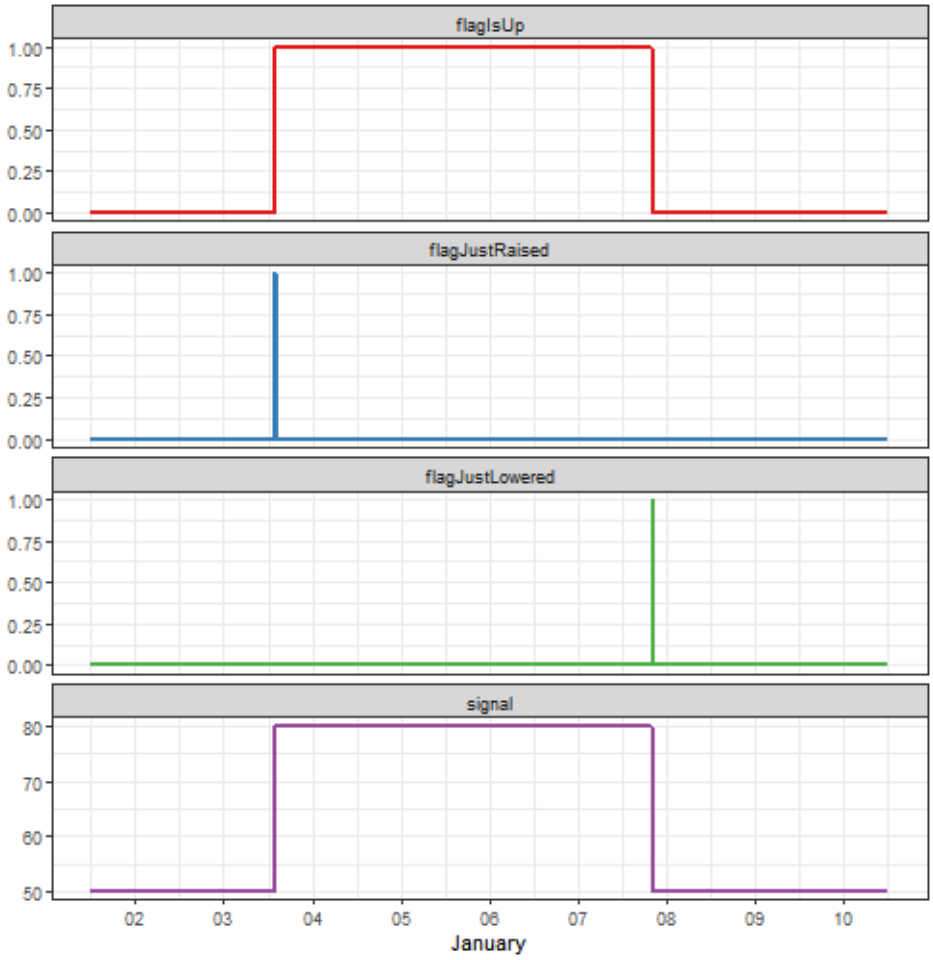


Figure 5: Non-circadian DateTimeSignal

The precise timing is readily revealed:

```
> rows 199 202
...
```

```
           dateTime flagIsUp flagJustRaised flagJustLowered signal iteration step
2001/01/03T13:30:00        0              0               0   50.0         1  198
2001/01/03T13:45:00        0              0               0   50.0         1  199
2001/01/03T14:00:00        1              1               0   80.0         1  200
2001/01/03T14:15:00        1              0               0   80.0         1  201

> rows 608 611
...
           dateTime flagIsUp flagJustRaised flagJustLowered signal iteration step
2001/01/07T19:45:00        1              0               0   80.0         1  607
2001/01/07T20:00:00        1              0               0   80.0         1  608
2001/01/07T20:15:00        0              0               1   50.0         1  609
2001/01/07T20:30:00        0              0               0   50.0         1  610
```

Alternatively, we can ignore the time of day all together, as shown in the date_time_signal3.box script:

```
DateTimeSignal signal {
  .beginDate  = 03/01/2001
  .endDate    = 07/01/2001
  .initialSignal = 50
  .signalInside  = 80
  .signalOutside = 50
  .circadian = FALSE
}
```

This gives us five full days. Here the signal is going *on* at midnight:

```
> rows 143 146
...
           dateTime flagIsUp flagJustRaised flagJustLowered signal iteration step
2001/01/02T23:30:00        0              0               0   50.0         1  142
2001/01/02T23:45:00        0              0               0   50.0         1  143
2001/01/03T00:00:00        1              1               0   80.0         1  144
2001/01/03T00:15:00        1              0               0   80.0         1  145
```

And here the signal is going *off* at midnight:

```
> rows 623 626
...
           dateTime flagIsUp flagJustRaised flagJustLowered signal iteration step
2001/01/07T23:30:00        1              0               0   80.0         1  622
2001/01/07T23:45:00        1              0               0   80.0         1  623
2001/01/08T00:00:00        0              0               1   50.0         1  624
2001/01/08T00:15:00        0              0               0   50.0         1  625
```

1.13 Class: DayDegrees

1.13.1 Inputs

Parameter	Type	Default	Description
`T`	`double`	a) °C	
`timeStepDays`	`double`	b) `d`	
`resetTotal`	`bool`	`FALSE`	Reset `total` zero?
`isTicking`	`bool`	`TRUE`	Is the physiological time running? Else `step` will be zero
`T0`	`double`	`0.0` °C	Lower temperature threshold
`Topt`	`double`	`100.0` °C	Optimum temperate; linear decline from here to `Tmax`
`Tmax`	`double`	`100.0` °C	Upper temperature threshold

a) `weather[Tavg]`
b) `calendar[timeStepDays]`

1.13.2 Outputs

Parameter	Type	Default	Description
`step`	`double`	`0.0` `oD`	Increment in day-degrees
`total`	`double`	`0.0` `oD`	Total time since reset

1.13.3 Usage

`DayDegrees` computes linear day-degrees.

1.14 Class: DemandBudget

1.14.1 Inputs

Parameter	Type	Default	Description
`demandNet`	`double`	`0.0` `#prey`	Net demand
`demandResp`	`double`	`0.0` `#prey`	Respiration demand
`egested`	`double`	`0.0` `[0;1]`	Egested or unused fraction of prey
`conversionCost`	`double`	`0.0` `[0;1]`	Cost of converting prey into consumer)

1.14.2 Outputs

Parameter	Type	Default	Description
`demandGross`	`double`	`0.0` `#prey`	Gross demand including all expenditures

1.14.3 Usage

`DemandBudget` computes gross demand from energy budget.

1.15 Class: DewPoint

1.15.1 Inputs

Parameter	Type	Default	Description
`temperature`	`double`	`0.0` °C	Air temperature
`rh`	`double`	`0.0` `%`	Air relative humidity

1.15.2 Outputs

Parameter	Type	Default	Description
`value`	`double`	`0.0` °C	Dew point temperature

1.15.3 Usage

`DewPoint` computes dew point.

1.16 Class: Distribution

1.16.1 Inputs

Parameter	Type	Default	Description
`input`	`double`	`0.0`	the input port to anayse
`divisor`	`double`	`100.0`	Divisor applied to `sections
`sections`	`vec_int`	a)	Sections applied at the end of every iteration

Parameter	Type	Default	Description
steps	int	/.[steps]	

a) c(1,5,25,50,75,95,99)

1.16.2 Usage

Distribution computes distribution fractiles.

1.17 Class: FixedSignal

1.17.1 Inputs

Parameter	Type	Default	Description
initialSignal	double	0.0	Value of signal at reset
initialFlag	bool	FALSE	Value of flag at reset

1.17.2 Outputs

Parameter	Type	Default	Description
signal	double	0.0	Value of the signal
flagIsUp	bool	FALSE	Is the signal on?
flagJustRaised	bool	FALSE	Was the signal just set on?
flagJustLowered	bool	FALSE	Was the signal just set off?
value	double	0.0	Synonym for signal

1.17.3 Usage

FixedSignal boxes::FixedSignal fixes a signal at the initial value.

1.18 Class: FoodWeb

1.18.1 Inputs

Parameter	Type	Default	Description
timeStep	double	1.0	Simulation time step

1.18.2 Usage

FoodWeb computes food web acquisition among Predator/Prey and Parasite/Host objects.

1.19 Class: FoodWebCalcParasiteHost

1.19.1 Inputs

Parameter	Type	Default	Description
timeStep	double	a)	

a) ..[timeStep]

1.19.2 Usage

FoodWebCalcParasiteHost computes food web acquisition among Parasite/Host objects.

1.20 Class: FoodWebCalcPredatorPrey

1.20.1 Inputs

Parameter	Type	Default	Description
timeStep	double	a)	

a) ..[timeStep]

1.20.2 Usage

FoodWebCalcPredatorPrey computes food web acquisition among Predator/Prey objects.

1.21 Class: FunctionalResponse

1.21.1 Inputs

Parameter	Type	Default	Description
attacker	double	0.0 #predators	Attacker density
prey	double	0.0 #prey	Resource density
demandGross	double	0.0 #prey	Gross demand of attacker
attackRate	double	0.0	#attacks per #prey per #predator per time unit
timeStep	double	1.0 Time unit	Time step

1.21.2 Outputs

Parameter	Type	Default	Description
supplyGross	double	0.0 #prey	Gross supply (number of prey attacked)
propPreyAttacked	double	0.0 [0;1]	The proportion of prey attacked
searchEfficacy	double	0.0	Efficacy in finding prey [0;1]
sdRatioGross	double	0.0 [0;1]	Predator's gross supply/demand ratio

1.22 Class: Host

1.22.1 Inputs

Parameter	Type	Default	Description
density	double	0.0	Density
attackRate	double	0.0 per parasite per time unit	Attack rate
gainFactor	double	1.0 supply/ attacks	Conversion of resource into supply units

1.22.2 Outputs

Parameter	Type	Default	Description
propAttacked	double	0.0	Proportion of hosts attacked
numAttacked	double	0.0	Number of hosts attacked

1.22.3 Usage

`Host` defines the host parameters for interaction with the parent Parasite.

1.23 Class: Hump

1.23.1 Inputs

Parameter	Type	Default	Description
`x`	`double`	`0.0`	x-value at which to calculate curve value
`x0`	`double`	`0.0`	Beginning of curve
`x1`	`double`	`1.0`	End of curve
`ymin`	`double`	`0.0`	Minimum value of curve
`ymax`	`double`	`1.0`	Maximum value of curve
`ditch`	`bool`	`FALSE`	Is the curve a ditch?

1.23.2 Outputs

Parameter	Type	Default	Description
`value`	`double`	`0.0`	Curve value at x-value

1.23.3 Usage

You use the `Hump` box to produce a generic, symmetric hump-or ditch-shaped curve with set limits on the x- and y-axis. The curve is a fourth degree polynomium:

$$f_{\text{hump}}(x) = y_{\min} + g(x)$$
$$f_{\text{ditch}}(x) = y_{\max} - g(x)$$

where

$$g(x) = \begin{cases} \left((y_{\max} - y_{\min}) \left(4 \frac{(x-x_0)(x-x_1)}{(x_0-x_1)(x_1-x_0)} \right)^2, \text{for} \quad x \in [x_0; x_1] \right) \\ (0, \text{otherwise}) \end{cases}$$

The following script demonstrates the shapes that can be obtained:

```
// hump1.box
Simulation sim {
  .steps = temperature[steps]
  SequenceByStep temperature {
    .min = -5
    .max = 40
    .by = 0.5
  }
  Box humps {
    Hump hump {
      .x = temperature[value]
      .x0 = 12
      .x1 = 34
      .ymax = 100
    }
    Hump humpOffset {
      .x = temperature[value]
      .x0 = 12
      .x1 = 34
```

```
      .ymax = 100
      .ymin = 20
    }
    Hump ditch {
      .x = temperature[value]
      .x0 = 12
      .x1 = 34
      .ymax = 100
      .ditch = TRUE
    }
    Hump ditchOffset {
      .x = temperature[value]
      .x0 = 12
      .x1 = 34
      .ymax = 100
      .ymin = 20
      .ditch = TRUE
    }
  }
  OutputR {
    PageR {
      .xAxis = temperature[value]
      PlotR {
        .ports = humps/*[value]
        .ncol = 2
      }
    }
  }
}
```

Here is the output:

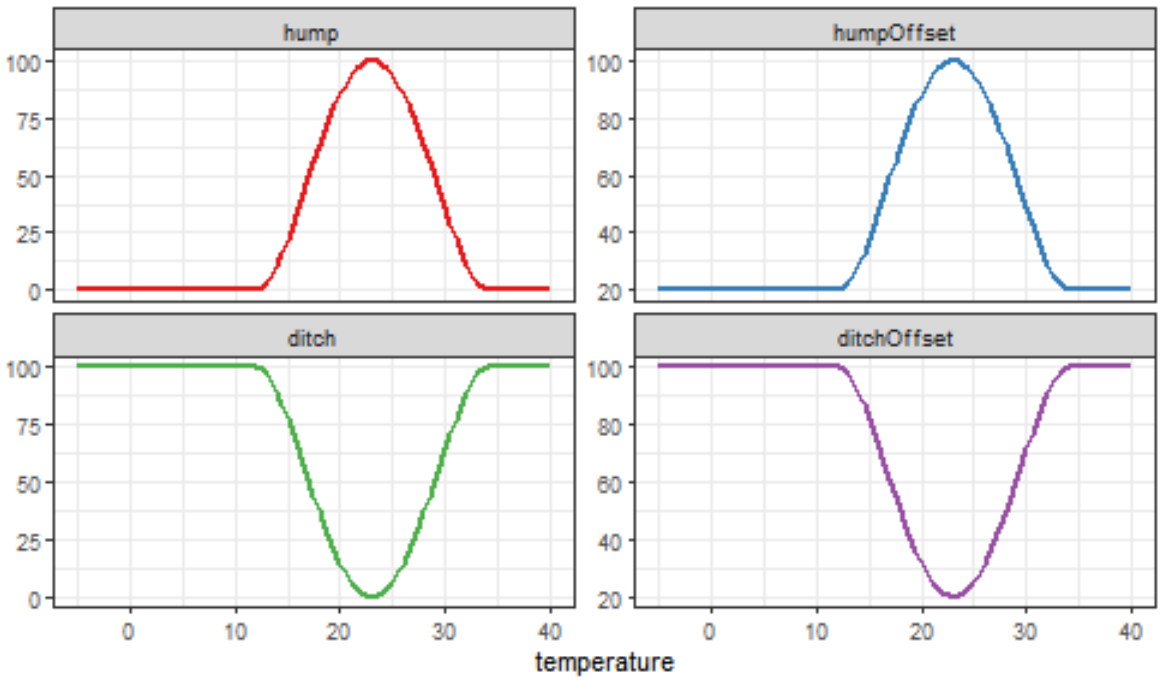


Figure 6: Hump example

The curve takes off exactly from its cloor or ceiling exactly at the limits `x0` and `x1` as you can see by the `rows` command:

```
> rows 34 37
...
temperature     hump humpOffset   ditch ditchOffset iteration step
       11.5      0.0       20.0   100.0       100.0         1   33
       12.0      0.0       20.0   100.0       100.0         1   34
       12.5 0.789307    20.6314 99.2107     99.3686         1   35
       13.0  3.01209    22.4097 96.9879     97.5903         1   36
> rows 77 80
...
temperature     hump humpOffset   ditch ditchOffset iteration step
       33.0  3.01209    22.4097 96.9879     97.5903         1   76
       33.5 0.789307    20.6314 99.2107     99.3686         1   77
       34.0      0.0       20.0   100.0       100.0         1   78
       34.5      0.0       20.0   100.0       100.0         1   79
```

1.24 Class: IgnoredBox

1.24.1 Usage

`IgnoredBox` is ignored during standard operations (`reset()`, `update()`, etc..

1.25 Class: LobryRossoFlandrois

1.25.1 Inputs

Parameter	Type	Default	Description
`T`	`double`	a) °C	
`timeStepDays`	`double`	b) `d`	
`resetTotal`	`bool`	`FALSE`	Reset `total` zero?
`isTicking`	`bool`	`TRUE`	Is the physiological time running? Else `step` will be zero
`Tmin`	`double`	`0.0` °C	Lower temperature threshold
`Topt`	`double`	`0.0` °C	Optimal temperature
`Tmax`	`double`	`0.0` °C	Upper temperature threshold
`maximum`	`double`	`0.0` /°C	Maximum development rate

a) `weather[Tavg]`
b) `calendar[timeStepDays]`

1.25.2 Outputs

Parameter	Type	Default	Description
`step`	`double`	`0.0` `PTU`	Increment in physiological time units (PTU)
`total`	`double`	`0.0` `PTU`	Total time since reset

1.25.3 Usage

`LobryRossoFlandrois` computes non-linear development rate per day.

1.26 Class: Maker

1.26.1 Inputs

Parameter	Type	Default	Description
`replicates`	`int`	`1`	Supply either, (1) the number of clones to replicate from child boxes;
`names`	`vec_string`	`null`	or (2) the names of clones;
`fileName`	`string`		or (3) a file; each line specifies a clone; column headings label inputs to each clone.

- Make numbered clones
- Make named clones
- Make clones from file

1.26.2 Usage

You use a `Maker` box if you want to make several copies (clones) of a box in the boxscript. The clones will all be exact copies of the original box (albeit with different names), including its port values and any child boxes, recursively (i.e. the clones are deep copies).

1.26.2.1 Make numbered clones

In the first example, we create want to create four box objets representing islands with populations of foxes and rabbits. The geolocation of the islands and animal densities should be random. Here is our first sketch of a model without the use of a `Maker`:

```
// islands1.box
Simulation sim {
  .iterations = 4
  .steps = 5
  Box random {
    RandomiserMonteCarlo randomiser {
    }
    RandomUniform latitude {
      .min = 10
      .max = 20
    }
    RandomUniform longitude {
      .min = -40
      .max = -30
    }
    RandomUniformInt initFox {
      .min = 10
      .max = 20
    }
    RandomUniformInt initRabbit {
      .min = 200
      .max = 1000
    }
  }
  Box island {
    &latitude = random/latitude[value]
    &longitude = random/longitude[value]
    Box fox {
      &value = random/initFox[value]
    }
```

```
    Box rabbit {
      &value = random/initRabbit[value]
    }
  }
  OutputText {
    .ports = sim[iteration] | island[*] | island/*[*]
    OutputSelector {
      .final = TRUE
    }
  }
}
```

With the `OutputSelector`'s `final` port set to `TRUE`, we get only one line in the output per iteration, as we can quickly see following `run` by `head`:

```
> run demo/maker/islands1.box
```

This boxscript will allow us to model on each island (given further elaborations of the model) *in sequence*. But if we wanted to model them *simultaneously*, maybe to include migration between the islands? For that we can use a `Maker` box:

```
// islands2.box
Simulation sim {
  .iterations = 4
  .steps = 5
  Box random {
    RandomiserMonteCarlo randomiser {
    }
    RandomUniform latitude {
      .min = 10
      .max = 20
    }
    RandomUniform longitude {
      .min = -40
      .max = -30
    }
    RandomUniformInt initFox {
      .min = 10
      .max = 20
    }
    RandomUniformInt initRabbit {
      .min = 200
      .max = 1000
    }
  }
  Maker archipelago {
    .replicates = 3
    Box isle {
      &latitude = random/latitude[value]
      &longitude = random/longitude[value]
      Box fox {
        &value = random/initFox[value]
      }
      Box rabbit {
        &value = random/initRabbit[value]
```

```
      }
    }
  }
  OutputText {
    .ports = sim[iteration] | archipelago/*[*] | archipelago/*/*[*]
    OutputSelector {
      .final = TRUE
    }
  }
}
```

Here, the `isle` is used as a template by the `Maker` (called `archipelago`) to make three clones (since `replicates = 3`, which are simply numbered. The `list` shows the model structure:

```
> list
```

If we run the model and look at the output, we are not quite there though. Take a look at the output (lines have been broken for easier reading):

```
> run demo/maker/islands2.box
```

In each of the four iterations, the three isles share their position (like being stacked), and they have the same densities of foxes and rabbits. We don't know the exact research questions addresses by this model but let's say that in each iteration, isles should be placed randomly and with different, random population densities. Then we need for each isle to draw its own random numbers:

```
// islands3.box
Simulation sim {
  .iterations = 4
  .steps = 5
  RandomiserMonteCarlo randomiser {
  }
  Maker archipelago {
    .replicates = 3
    Box isle {
      &latitude = ./latitude[value]
      &longitude = ./longitude[value]
      RandomUniform latitude {
        .min = 10
        .max = 20
      }
      RandomUniform longitude {
        .min = -40
        .max = -30
      }
      Box fox {
        &value = ./init[value]
        RandomUniformInt init {
          .min = 10
          .max = 20
        }
      }
      Box rabbit {
        &value = ./init[value]
        RandomUniformInt init {
          .min = 200
```

```
          .max = 1000
        }
      }
    }
  }
  OutputText {
    .ports = sim[iteration] | archipelago/*[*] | fox[value] | rabbit[value]
    OutputSelector {
      .final = TRUE
    }
  }
}
```

If we inspect `isle0`, we can see that is equipped with its own random numbers:

```
> list isle0
```

This is confirmed further by the output of the model. Now the isles are different in all aspects in every iteration:

```
> head
```

Note the there can only be one randomiser box on a boxscript. All the random variables in a boxscript draw numbers from this one source.

1.26.2.2 Make named clones

To continue the example above, let's say that we want to model some concrete islands with known positions but still with random population densities. We can achieve this by supplying the `Maker` box with a vector of `names` in stead of just the number of replicates. In this example, we include only to islands:

```
// islands4.box
Simulation sim {
  .iterations = 4
  .steps = 5
  RandomiserMonteCarlo randomiser {
  }
  Maker azores {
    .names = c("Faial", "Flores")
    Box isle {
      &latitude  = if (name(.[]) == "Faial") then 38.58 else 39.46
      &longitude = if (name(.[]) == "Faial") then 28.70 else 31.13
      Box fox {
        &value = ./init[value]
        RandomUniformInt init {
          .min = 10
          .max = 20
        }
      }
      Box rabbit {
        &value = ./init[value]
        RandomUniformInt init {
          .min = 200
          .max = 1000
        }
      }
    }
  }
  OutputText {
```

```
    .ports = sim[iteration] | archipelago/*[*] | fox[value] | rabbit[value]
    OutputSelector {
      .final = TRUE
    }
  }
}
```

As you can see, we got the wished for structure of the model:

```
> list
```

1.26.2.3 Make clones from file

To do…

1.27 Class: Maximum

1.27.1 Inputs

Parameter	Type	Default	Description
values	vec_double	null	Input values

1.27.2 Outputs

Parameter	Type	Default	Description
value	double	0.0	Maximum of inputs; zero if no inputs
count	int	0	Number of inputs

1.28 Class: MaximumAt

1.28.1 Inputs

Parameter	Type	Default	Description
step	int	/.[step]	
track	double	0.0	Value to track for max value

1.28.2 Outputs

Parameter	Type	Default	Description
at	int	0	Step in which maximum value was attained
value	double	0.0	The maximum value attained

1.28.3 Background

If you need to summarise the trend of a port by the maximum value it attained and in which step it happened, you can use a `MaximumAt` box. The interface is simple. Just watch the example below. If you run the simulation for several iterations, the outputs `at` and `max` may differ between iterations.

1.28.4 Usage

We will use a Hump box for demonstration since we now the position of its maximum value:

```
// maximum_at1.box
Simulation sim {
  .steps = 50
  Hump hump {
    .x = sim[step]
```

```
    .x0 = 20
    .x1 = 48
    .ymax = 150
  }
  MaximumAt maxValue {
    .track = hump[value]
  }
  OutputR {
    PageR {
      PlotR {
        .ports = hump[value] | maxValue[output::*]
        .ggplot = "geom_vline(xintercept=34, colour='black', linetype='dashed')"
      }
    }
  }
}
```

With this parameter setting we expect to find the maximum in the point (34, 150). This is confirmed when we run the script,

```
> run demo/maximum_at/maximum_at1.box
```

and study the output:

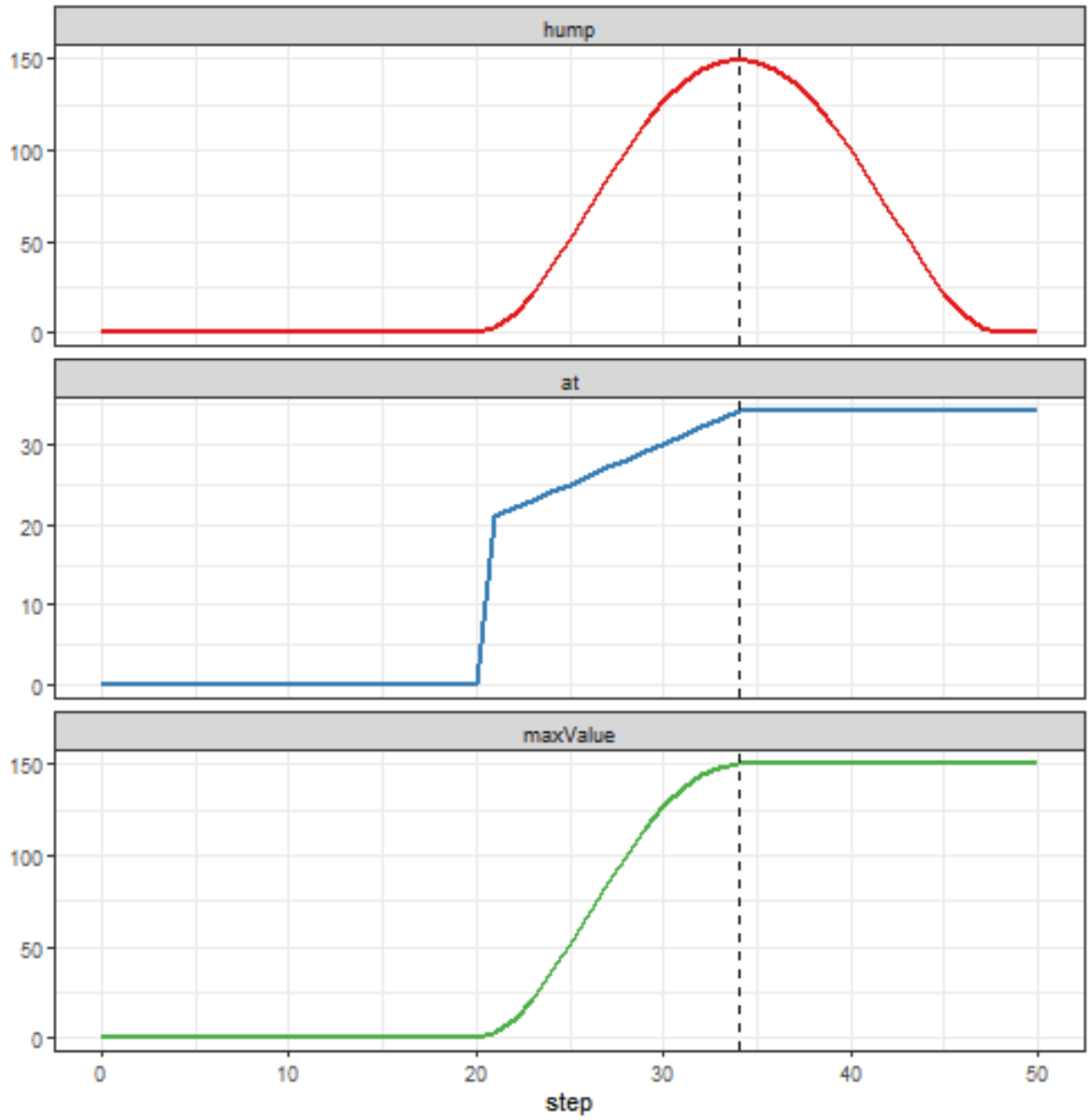


Figure 7: MaximumAt example

Further confirmation can be gotten by the tail command:

```
> tail 3
```

1.29 Class: Mean

1.29.1 Inputs

Parameter	Type	Default	Description
`values`	`vec_double`	`null`	Input values

1.29.2 Outputs

Parameter	Type	Default	Description
`value`	`double`	`0.0`	Average of inputs; zero if no inputs
`count`	`int`	`0`	Number of inputs

1.30 Class: Message

1.30.1 Inputs

Parameter	Type	Default	Description
`text`	`string`		Text message to print
`when`	`string`		In which step to print the message (reset, initialize, etc.)

1.31 Class: Minimum

1.31.1 Inputs

Parameter	Type	Default	Description
`values`	`vec_double`	`null`	Input values

1.31.2 Outputs

Parameter	Type	Default	Description
`value`	`double`	`0.0`	Minimum of inputs; zero if no inputs
`count`	`int`	`0`	Number of inputs

1.32 Class: Multiplum

1.32.1 Inputs

Parameter	Type	Default	Description
`factors`	`vec_double`	`null`	Input factors

1.32.2 Outputs

Parameter	Type	Default	Description
`value`	`double`	`0.0`	The multiplum

1.33 Class: NumberedFile

1.33.1 Inputs

Parameter	Type	Default	Description
`fileName`	`string`		File name with extension, with/without a path
`evaluationOrder`	`int`	`0`	Number to append to file name
`numberWidth`	`int`	`0`	Number will be left-filled with zero up to this width

1.33.2 Outputs

Parameter	Type	Default	Description
`value`	`string`		File name with number

1.34 Class: OffsetDateTime

1.34.1 Inputs

Parameter	Type	Default	Description
dateTime	datetime	null	Date-time that will be offset
days	int	0 d	Offset (negative or positive)
hours	int	0 h	Offset (negative or positive)
minutes	int	0 m	Offset (negative or positive)

1.34.2 Outputs

Parameter	Type	Default	Description
value	datetime	null	Offset date-time

1.35 Class: OnOff

1.35.1 Inputs

Parameter	Type	Default	Description
x	double	0.0	x-value at which to calculate value
xOn	double	0.0	Lower threshold
xOff	double	0.0	Upper threshold
valueOn	double	1.0	Value inside [xOn,xOff] interval
valueOff	double	0.0	Value outside [xOn,xOff] interval
isSticky	bool	FALSE	Should remain on when switched on?

1.35.2 Outputs

Parameter	Type	Default	Description
value	double	0.0	Current on or off value
isOn	bool	FALSE	Is switch on?

1.35.3 Usage

`OnOff` is a container of boxes and ports.

1.36 Class: OnOffByDate

1.36.1 Inputs

Parameter	Type	Default	Description
x	date	null date	x-value at which to calculate value
xOn	baredate	null Bare date	Lower threshold
xOff	baredate	null Bare date	Upper threshold
valueOn	double	1.0	Value inside [xOn,xOff] interval
valueOff	double	0.0	Value outside [xOn,xOff] interval
isSticky	bool	FALSE	Should remain on when switched on?

1.36.2 Outputs

Parameter	Type	Default	Description
value	double	0.0	Current on or off value
isOn	bool	FALSE	Is switch on?

1.36.3 Usage

`OnOffByDate` is a container of boxes and ports.

1.37 Class: OutputR

1.37.1 Inputs

Parameter	Type	Default	Description
keepPlots	bool	TRUE	Keep previous plots before showing new plots in R?
clearMemory	bool	FALSE	Clear R memory?
showPlots	bool	TRUE	Show plots in R?
showLines	int	0	Number of lines to show at the prompt
popUp	bool	FALSE	Show plots in pop-up windows?
width	double	7.0	Width of pop-up windows (only used if popUp is set)
height	double	7.0	Height of pop-up windows (only used if popUp is set)
maximizeWindow	bool	FALSE	Maximize plot window size?
fontSize	double	0.0	Only used if >0
lineWidth	double	1.0	Width of lines in plots
plotAsList	bool	FALSE	Put plots into an R list?
saveDataFrame	bool	FALSE	Save output as R data frame?
skipFormats	bool	a)	
code	string		R code to be run before 'scripts'
scripts	vec_string	null	R scripts to be run at the end
plotTypes	vec_string	b)	
clearPlots	bool	FALSE	Deprecated
hasError	bool	c)	

a) `OutputWriter::*[skipFormats]`
b) `PlotR::*[type]`
c) `/.[hasError]`

1.37.2 Outputs

Parameter	Type	Default	Description
ports	path	a)	Path to all ports used by my pages and plots
numPages	int	0	Number of pages in this output

a) `./PageR::*[xAxis]|./*/PlotR::*[ports]`

Note the side effects: writes an R script to the output folder copies an R script to the clipboard

1.38 Class: OutputSelector

1.38.1 Inputs

Parameter	Type	Default	Description
`beginStep`	`int`	`0 int`	Output is written when this step is reached
`beginDateTime`	`datetime`	`null DateTime`	Output is written when this time point is reached (optional)
`step`	`int`	`/.[step]`	
`dateTime`	`datetime`	a)	
`period`	`int`	`1`	If >1: A row of output will be produced with this period
`summary`	`string`	`"average"`	Either `"average"` or `"current"`; how values are summarized over the given `period`
`final`	`bool`	`FALSE`	Overrules `period`
`useLocalDecimalChar`	`bool`	`FALSE`	Use local decimal character in output?
`skipFormats`	`bool`	b)	

a) `if exists(Calendar::*[dateTime]) then Calendar::*[dateTime] else .[beginDateTime]`
b) `! exists(OutputR::*[skipFormats])`

1.38.2 Outputs

Parameter	Type	Default	Description
`isActive`	`bool`	`FALSE`	Should output be written?
`isSkipping`	`bool`	`FALSE`	Are lines being skipped?

- Usage
- Skip a number of lines
- Skip lines until a certain time
- Periodic output
- Final output

1.38.3 Usage

The `OutputSelector` box determines when output should be produced by the `OutputWriter` box. Hence, if you do not include an `OutputSelector` box in your boxscript then the `OutputWriter` box will create one for you. The default behaviour of `OutputSelector` is to write one line of output for every *reset* and *update* step during the simulation (see Computationel model). The `useLocalDecimalChar` and `skipFormats` define different features of the output file as explained for OutputWriter.

If you want to change the default behaviour of `OutputSelector`, you must include an `OutputSelector` box in your boxscript. The right place to put it is as a child box of the `OutputText` (as exemplified below) or `OutputR` box, since `OutputSelector` affects the output produced by `OutputText` and `OutputR`.

1.38.4 Skip a number of lines

You can set the number `steps` and `iterations` a simulation is running in the Simulation object. That will by itself result in `iterations` * (`steps` + 1) lines in the output file (adding one for each iteration because a line is written also in the reset step). You can skip a number of initial lines in each iteration by setting the `beginStep` of the `OutputSelector` box. Maybe the model needs some warming up before it produces reliable or relevant output. Here is one simple example:

```
// output_selector1.box
Simulation sim {
  .iterations = 2
```

```
  .steps = 12
  Calendar calendar {
  }
  OutputText {
    .ports = calendar[date]
    OutputSelector {
      .beginStep = 10
    }
  }
}
```

When you run it

```
> run demo/output_selector/output_selector1.box
```

and subsequently inspect the few lines produced with the `head` command, you won't be surprised:

```
> head
```

Well, unless you had not anticipated that `beginStep` is acting on every `iteration`.

1.38.5 Skip lines until a certain time

As an alternative to `beginStep`, you can set `beginDateTime` to commence output at a certain date-time. You can combine them both, which will postpone output until both a certain step and a certain date-time is reached, though I never found the need to do so. Here is a simple example, working in 5-minute time steps

```
// output_selector2.box
Simulation sim {
  .iterations = 2
  Calendar calendar {
    .begin = 01/08/2022T12:00
    .end = 01/08/2022T12:40
    .timeStep = 5
    .timeUnit = "m"
  }
  OutputText {
    .ports = calendar[dateTime]
    OutputSelector {
      .beginDateTime = 01/08/2022T12:30
    }
  }
}
```

Run the simulation

```
run demo/output_selector/output_selector2.box
```

and again use `head` to see the expected output:

```
> head
```

1.38.6 Periodic output

You can write only every other line by setting `period = 2`, every third line by `period = 3`, and so forth. This can be combined with `beginStep` and `beginDateTime` as you wish:

```
// output_selector3.box
Simulation sim {
  .iterations = 2
  .steps = 25
```

```
  Calendar calendar {
  }
  OutputText {
    .ports = calendar[date]
    OutputSelector {
      .beginStep = 10
      .period = 6
    }
  }
}
```

Notice that the final output written for each iteration is in step 22, even though an iteration lasts 25 steps. Note also that the period counts from the first line written in each iteration (in step 10), not from the beginning of the simulation in each iteration (step 0):

```
> head
```

1.38.7 Final output

If you want only the final values to appear in the output, i.e. those in the last step, then you set `final` to `TRUE`. This will overrule any of the settings on `beginStep`, `beginDateTime` and `period`. Here is a simple boxscript:

```
// output_selector4.box
Simulation sim {
  .iterations = 3
  .steps = 25
  Calendar calendar {
  }
  OutputText {
    .ports = calendar[date]
    OutputSelector {
      .final = TRUE
    }
  }
}
```

When you run the script,

```
> run demo/output_selector/output_selector4.box
```

and look at the output, you will find that only the final step 25 of each of the three iterations appears in the output:

```
> head 20
```

If you run an uncertainty or sensitivity analysis, you will usually set `final` to `TRUE`.

1.39 Class: OutputText

1.39.1 Inputs

Parameter	Type	Default	Description
`ports`	`path`	`null`	Path to port(s) to output

1.40 Class: OutputWriter

1.40.1 Inputs

Parameter	Type	Default	Description
showPorts	bool	FALSE	Shows a table of the ports to be written
ports	path	a)	
skipFormats	bool	b)	
useLocalDecimalChar	bool	c)	
isSkipping	bool	d)	
isActive	bool	e)	
period	int	f)	
summary	string	g)	

a) `/.[iteration]|/.[step]|OutputText::*[ports]|OutputR::*[ports]`
b) `OutputSelector::*[skipFormats]`
c) `OutputSelector::*[useLocalDecimalChar]`
d) `OutputSelector::*[isSkipping]`
e) `OutputSelector::*[isActive]`
f) `OutputSelector::*[period]`
g) `OutputSelector::*[summary]`

1.40.2 Outputs

Parameter	Type	Default	Description
filePath	string		Name of output file including absolute path
decimalChar	string		Decimal character used in output

- Usage
- Output folder
- Output file format

1.40.3 Usage

You'll never need to include an `OutputWriter` box in a boxscript explicitly. It will be created automaticall by boxes that need it, such as OutputR and OutputText. It is the sole responsibility of `OutputWriter` to write the simulation output to a file. All of its inputshas been given default expressions, which look up the relevant data in other boxes. You set *what* to change what is written in the OutputR or OutputText box and you can change *how* it is written in the OutputSelector box.

`OutputWriter` depends much on `OutputSelector`, in fact, so much that if no `OutputSelector` box is present in the boxscript then the `OutputWriter` box will create an `OutputSelector` as a child box.

The sun1.box ▾ script, for example, includes an `OutputR` box explicitly in the script. In a chain reaction, the `OutputR` box creates an `OutputWriter` box, which again creates an `OutputSelector` box:

```
> load demo/sun/sun1.box
```

Most often you are not interested in the output file, as Universal Simulator does its best to handle all output automatically behind the scenes but, anyway, here are the details.

1.40.4 Output folder

The output file will be written to your output folder, which you can locate with the get folders command:

```
> get folders
```

The name of the output file will be the same as the boxscript that produced it, just with a four-digit running number at the end. As a pure text file, its suffix (also known as its file type) is "txt". The file consists of tab-separated columns, separated by tab characters (no, you cannot change the separator character to something else than a tab).

The output folder will grow slowly for every run you make. Therefore, you should sometimes empty it.

1.40.5 Output file format

The first line contains unique column names constructed from the names of the `ports` included. The column names have been shortened as much as possible while keeping them unique. Ports named `value` will have a column name without the port part if possible; they will be named after their parent box instead ("value" is not an informative label).

If `skipFormats` is `FALSE`, which it defaults to because that's the default value imported from OutputSelector, then the second line in the output file will be information about the format of the values in the column. These format specifications are used when the file is read into R as a data frame. If you want to do all the handling of the output file yourself then set `skipFormats` to `TRUE` in the `OutputSelector` box.

If you want to use the decimal character specific to the language settings on your computer then set `useLocalDecimalChar` to `TRUE` in the `OutputSelector` box; otherwise a decimal point will be used.

1.41 Class: PageR

1.41.1 Inputs

Parameter	Type	Default	Description
`xAxis`	`path`	`/.[step]`	Port(s) on x-axis
`title`	`string`		Title shown on page
`ncol`	`int`	`-1`	No. of columns to arrange plots in
`nrow`	`int`	`-1`	No. of rows to arrange plots in
`commonLegend`	`bool`	`FALSE`	Collate legends of blots into one common legend
`legendPosition`	`string`	`"bottom"`	If 'commonLegend' then place legend here
`width`	`double`	`..[width]`	
`height`	`double`	`..[height]`	
`maximizeWindow`	`bool`	a)	
`fontSize`	`double`	b)	
`plotAsList`	`bool`	c)	
`popUp`	`bool`	`..[popUp]`	
`numPages`	`int`	d)	
`layouts`	`vec_string`	e)	

a) `..[maximizeWindow]`
b) `..[fontSize]`
c) `..[plotAsList]`
d) `..[numPages]`
e) `./PlotR::*[layout]`

1.42 Class: Parasite

1.42.1 Inputs

Parameter	Type	Default	Description
`density`	`double`	`0.0`	Density

Parameter	Type	Default	Description
demand	double	0.0	Amount of resource needed

1.42.2 Outputs

Parameter	Type	Default	Description
supply	double	0.0	Total amount of resource acquired
sdRatio	double	0.0 [0;1]	Total supply/demand ratio

1.42.3 Usage

Parasite defines the demand and supply of a parasite.

1.43 Class: PlotR

1.43.1 Inputs

Parameter	Type	Default	Description
ports	path	null	Port(s) on y-axis
layout	string	"facetted"	Either "merged" or "facetted"
type	string	"default"	Type of plot (default|density|histogram(nbins)|SobolConvergence|SobolIndices)
guideTitle	string		Title of guide legends
ggplot	string		R code that will be added to the ggplot
end	string		Deprecated
endCode	string		Deprecated
maxData	int	0	Max. number of data rows to plot; ignored if zero
ncol	int	-1	Number of columns in arrangement of plots; -1 keeps default
nrow	int	-1	Number of rows in arrangement of plots; -1 keeps default
direction	string	"row"	Fill in plots by 'row' or 'col'
iteration	int	a)	
xAxis	path	..[xAxis]	
width	double	..[width]	
height	double	..[height]	
fontSize	int	b)	
plotAsList	bool	c)	

a) /.[iteration]
b) ..[fontSize]
c) ..[plotAsList]

1.44 Class: PopUp

1.44.1 Inputs

Parameter	Type	Default	Description
title	string		Title

Parameter	Type	Default	Description
text	string		Short informative text
details	string		Further details
buttons	vec_string	c("OK")	Vector of buttons to show: (OK Yes No)
defaultButton	string	"OK"	Button assumed when Return is typed
escapeButton	string	"OK"	Button assumed when Esc is typed
icon	string	a)	Icon to show
when	string	b)	In which step to show message, e.g., "initialize" or "debrief"
show	bool	TRUE	Determines whether pop-up will be shown

a) "information"
b) "initialize"

1.44.2 Outputs

Parameter	Type	Default	Description
answer	string		Name of button pushed
accepted	bool	FALSE	Question accepted?

1.45 Class: Population

1.45.1 Inputs

Parameter	Type	Default	Description
initial	double	100.0	
ageIncrement	double	0.0	
bufferSize	int	100	
firstCohortGain	double	0.0	
cohortsGain	vec_double	null	
cohortsLoss	vec_double	null	

1.45.2 Outputs

Parameter	Type	Default	Description
lastCohortSpill	double	0.0	
cohorts	vec_double	null	
age	vec_double	null	

1.45.3 Usage

Population models population dynamics by cohorts (experimental).

1.46 Class: Predator

1.46.1 Inputs

Parameter	Type	Default	Description
density	double	0.0	Density
demand	double	0.0	Amount of resource needed

1.46.2 Outputs

Parameter	Type	Default	Description
supply	double	0.0	Total amount of resource acquired
sdRatio	double	0.0 [0;1]	Total supply/demand ratio

1.46.3 Usage

Predator defines the demand and supply of a predator.

1.47 Class: Prey

1.47.1 Inputs

Parameter	Type	Default	Description
density	double	0.0	Density
attackRate	double	0.0 per predator per time unit	Attack rate
gainFactor	double	1.0 supply/ resource	Conversion of resource into supply units

1.47.2 Outputs

Parameter	Type	Default	Description
loss	double	0.0	Amount of prey lost to predators
mortality	double	0.0	Mortality caused by predators

1.47.3 Usage

Prey defines the prey parameters for interaction with the parent Predator.

1.48 Class: PrioritySignal

1.48.1 Inputs

Parameter	Type	Default	Description
initialSignal	double	0.0	Value of signal at reset
initialFlag	bool	FALSE	Value of flag at reset
myFlags	vec_bool	a)	
mySignals	vec_double	b)	
reverseOrder	bool	FALSE y|n	Find first signal!=0 from top (false) or bottom (true)?

a) ./BaseSignal::*[flagIsUp]
b) ./BaseSignal::*[signal]

1.48.2 Outputs

Parameter	Type	Default	Description
signal	double	0.0	Value of the signal
flagIsUp	bool	FALSE	Is the signal on?
flagJustRaised	bool	FALSE	Was the signal just set on?
flagJustLowered	bool	FALSE	Was the signal just set off?

Parameter	Type	Default	Description
value	double	0.0	Synonym for signal

1.49 Class: ProportionalSignal

1.49.1 Inputs

Parameter	Type	Default	Description
initialSignal	double	0.0	Value of signal at reset
initialFlag	bool	FALSE	Value of flag at reset
input	double	25.0	Value determining the signal
threshold	double	25.0	Input threshold above which the signal is changing
thresholdBand	double	5.0	Interval of the input over which the signal is changing
minSignal	double	0.0	Minimum possible signal
maxSignal	double	100.0	Maximum possible signal
increasingSignal	bool	TRUE y|n	Is the signal increasing inside the threshold band?

1.49.2 Outputs

Parameter	Type	Default	Description
signal	double	0.0	Value of the signal
flagIsUp	bool	FALSE	Is the signal on?
flagJustRaised	bool	FALSE	Was the signal just set on?
flagJustLowered	bool	FALSE	Was the signal just set off?
value	double	0.0	Synonym for signal

1.50 Class: RandomBinomial

1.50.1 Inputs

Parameter	Type	Default	Description
P	double	0.5 [0;1]	Probability of event
drawAtInitialize	bool	a)	
drawAtReset	bool	b)	
drawAtUpdate	bool	c)	
min	bool	FALSE	Not used
max	bool	TRUE	Not used

a) ancestors::*/RandomiserBase::*[drawAtInitialize]
b) ancestors::*/RandomiserBase::*[drawAtReset]
c) ancestors::*/RandomiserBase::*[drawAtUpdate]

1.50.2 Outputs

Parameter	Type	Default	Description
value	bool	FALSE	The most recently drawn value

1.50.3 Usage

`RandomBinomial` produces numbers from the binomial distribution. `value` will be `TRUE` with probability `P`.

```
// random_binomial1.box
Simulation sim {
  .steps = 100
  Box random {
    RandomiserMonteCarlo randomiser {
      .drawAtUpdate = TRUE
    }
    RandomBinomial x {
      .P = 0.6
    }
    RandomBinomial y {
      .P = 0.9
    }
  }
  Box positive {
    Accumulator x {
      .change = if (random/x[value]) then 1 else 0
    }
    Accumulator y {
      .change = if (random/y[value]) then 1 else 0
    }
  }
  OutputR {
    OutputText {
      .ports = random/*[value] | positive/*[value]
    }
  }
}
```

Run the boxscript:

```
> run demo/random_binomial/random_binomial1.box
```

The final lines of the output show how `x` approaches 60% and `y` approaches 90%, as expected:

```
> tail
```

1.51 Class: RandomLogNormal

1.51.1 Inputs

Parameter	Type	Default	Description
`P`	`double`	`0.95 [0;1]`	Range of the distribution covered by the [minValue;maxValue] range
`drawAtInitialize`	`bool`	a)	
`drawAtReset`	`bool`	b)	
`drawAtUpdate`	`bool`	c)	
`min`	`double`	`0.01`	Minimum value (included)
`max`	`double`	`1.0`	Maximum value (excluded)

a) `ancestors::*/RandomiserBase::*[drawAtInitialize]`
b) `ancestors::*/RandomiserBase::*[drawAtReset]`
c) `ancestors::*/RandomiserBase::*[drawAtUpdate]`

1.51.2 Outputs

Parameter	Type	Default	Description
value	double	0.0	The most recently drawn value

1.52 Class: RandomLogUniform

1.52.1 Inputs

Parameter	Type	Default	Description
P	double	0.95 [0;1]	Range of the distribution covered by the [minValue;maxValue] range
drawAtInitialize	bool	a)	
drawAtReset	bool	b)	
drawAtUpdate	bool	c)	
min	double	0.01	Minimum value (included)
max	double	1.0	Maximum value (excluded)

a) ancestors::*/RandomiserBase::*[drawAtInitialize]
b) ancestors::*/RandomiserBase::*[drawAtReset]
c) ancestors::*/RandomiserBase::*[drawAtUpdate]

1.52.2 Outputs

Parameter	Type	Default	Description
value	double	0.0	The most recently drawn value

1.53 Class: RandomNormal

1.53.1 Inputs

Parameter	Type	Default	Description
P	double	0.95 [0;1]	Range of the distribution covered by the [minValue;maxValue] range
drawAtInitialize	bool	a)	
drawAtReset	bool	b)	
drawAtUpdate	bool	c)	
min	double	0.0	Minimum value (included)
max	double	1.0	Maximum value (excluded)

a) ancestors::*/RandomiserBase::*[drawAtInitialize]
b) ancestors::*/RandomiserBase::*[drawAtReset]
c) ancestors::*/RandomiserBase::*[drawAtUpdate]

1.53.2 Outputs

Parameter	Type	Default	Description
value	double	0.0	The most recently drawn value

1.54 Class: RandomUniform

1.54.1 Inputs

Parameter	Type	Default	Description
P	double	0.95 [0;1]	Range of the distribution covered by the [minValue;maxValue] range
drawAtInitialize	bool	a)	
drawAtReset	bool	b)	
drawAtUpdate	bool	c)	
min	double	0.0	Minimum value (included)
max	double	1.0	Maximum value (excluded)

a) ancestors::*/RandomiserBase::*[drawAtInitialize]
b) ancestors::*/RandomiserBase::*[drawAtReset]
c) ancestors::*/RandomiserBase::*[drawAtUpdate]

1.54.2 Outputs

Parameter	Type	Default	Description
value	double	0.0	The most recently drawn value

1.55 Class: RandomUniformInt

1.55.1 Inputs

Parameter	Type	Default	Description
P	double	0.95 [0;1]	Range of the distribution covered by the [minValue;maxValue] range
drawAtInitialize	bool	a)	
drawAtReset	bool	b)	
drawAtUpdate	bool	c)	
min	int	0	Minimum value (included)
max	int	1	Maximum value (included)

a) ancestors::*/RandomiserBase::*[drawAtInitialize]
b) ancestors::*/RandomiserBase::*[drawAtReset]
c) ancestors::*/RandomiserBase::*[drawAtUpdate]

1.55.2 Outputs

Parameter	Type	Default	Description
value	int	0	The most recently drawn value

1.56 Class: RandomiserMonteCarlo

1.56.1 Inputs

Parameter	Type	Default	Description
iteration	int	a)	
iterations	int	b)	
doSensitivityAnalysis	bool	FALSE	Carry out a sensitivity analysis?

Parameter	Type	Default	Description
bootstrapSize	int	1000	Size of bootstrap sample (cheap in computation time); only used in sensitivity analysis
seed	int	0	Seed for random numbers; if this is zero a random seed value will be used
drawAtInitialize	bool	FALSE	Draw a value when a box is initialized?
drawAtReset	bool	TRUE	Draw a value when a box is reset?
drawAtUpdate	bool	FALSE	Draw a value when a box is updated?

a) /.[iteration]
b) /.[iterations]

1.56.2 Outputs

Parameter	Type	Default	Description
numVariables	int	0 [0;inf)	Number of variables which are randomised

See RandomiserSobolSequence.

1.57 Class: RandomiserSobolSequence

1.57.1 Inputs

Parameter	Type	Default	Description
iteration	int	a)	
iterations	int	b)	
doSensitivityAnalysis	bool	FALSE	Carry out a sensitivity analysis?
bootstrapSize	int	1000	Size of bootstrap sample (cheap in computation time); only used in sensitivity analysis
seed	int	0	Seed for random numbers; if this is zero a random seed value will be used
drawAtInitialize	bool	FALSE	Draw a value when a box is initialized?
drawAtReset	bool	TRUE	Draw a value when a box is reset?
drawAtUpdate	bool	FALSE	Draw a value when a box is updated?

a) /.[iteration]
b) /.[iterations]

1.57.2 Outputs

Parameter	Type	Default	Description
numVariables	int	0 [0;inf)	Number of variables which are randomised

1.57.3 Usage

You can use RandomiserMonteCarlo for stochastic modelling and uncertainty analysis, but RandomiserStratified will explore the parameter space more efficiently. For sensitivity analysis, RandomiserSobolSequence is the preferred choice because it is the most effective of the three in covering all of the parameter space.

The randomiser7.box ▾ script shows the differences, subtle but important, between the three randomisers. The boxscript breaks the general rule not to put more than one randomiser box into a script. The only good reason would be for a direct comparison like here. Each of the three randomisers is the source of random numbers used to generate random numbers for two variables x and y, both with a uniform distribution of numbers in the interval [20;40) (see RandomUniform).

```
// randomiser7.box

Simulation sim {
  .iterations = 32
  .steps = 1
  .silent = TRUE
  Box monteCarlo {
    RandomiserMonteCarlo randomiser {
    }
    RandomUniform x {
      .min = 20
      .max = 40
    }
    RandomUniform y {
      .min = 20
      .max = 40
    }
  }
  Box stratified {
    RandomiserStratified randomiser {
    }
    RandomUniform x {
      .min = 20
      .max = 40
    }
    RandomUniform y {
      .min = 20
      .max = 40
    }
  }
  Box sobolSequence {
    RandomiserSobolSequence randomiser {
    }
    RandomUniform x {
      .min = 20
      .max = 40
    }
    RandomUniform y {
      .min = 20
      .max = 40
    }
  }
  OutputR {
    .scripts = "randomiser7.R"
    OutputSelector {
      .final = TRUE
    }
    OutputText {
      .ports = sim/*/*[value]
    }
  }
}
```

Run the boxscript,

```
>run demo/randomiser/randomiser7.box
```

and study the output:

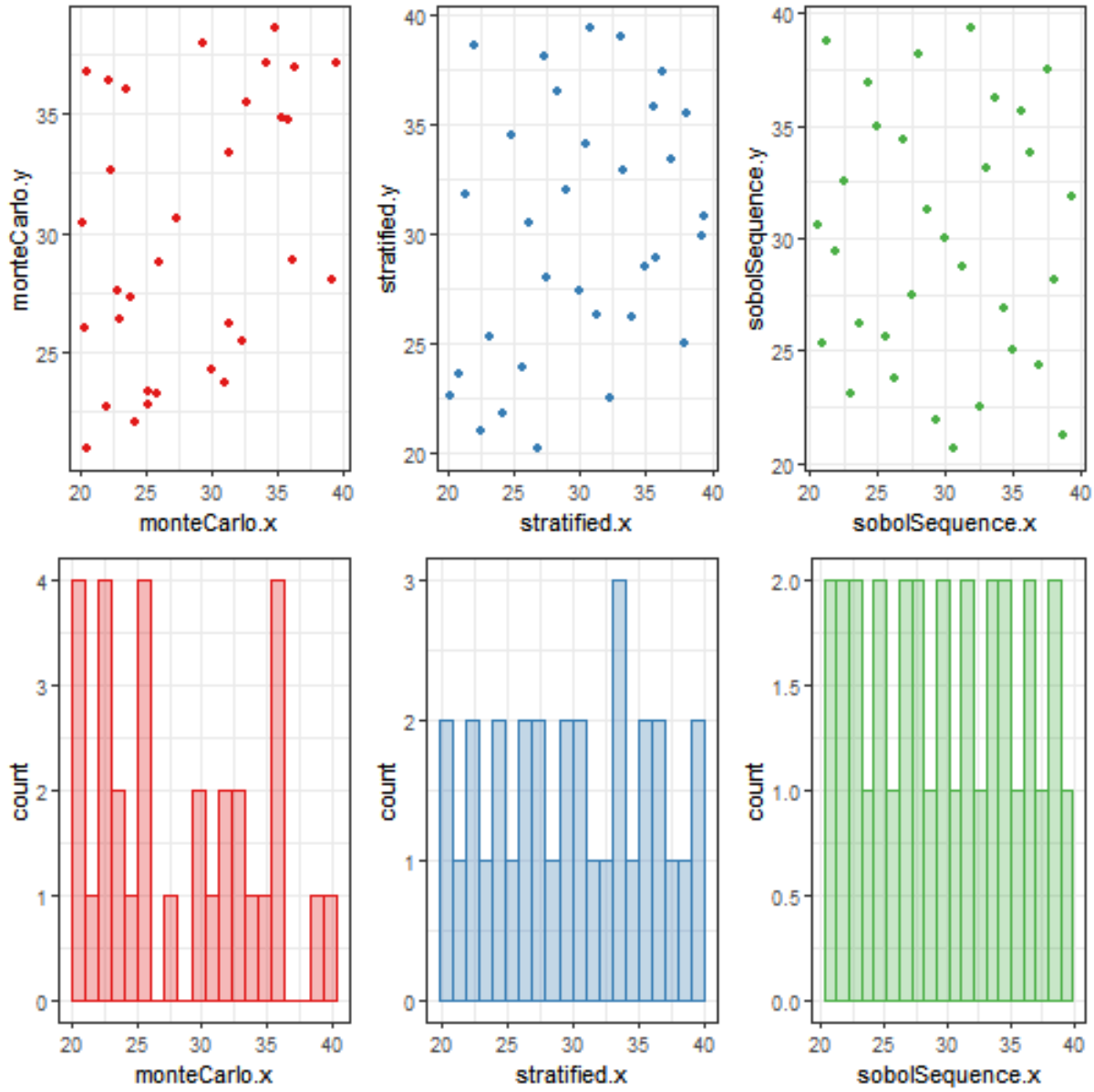


Figure 8: RandomiserSobolSequence example

The three ransomisers worked like this to generate random numbers:

- `RandomiserMonteCarlo` picked 32 numbers at random from the interval [20;40) for both `x` and `y`.
- `RandomiserStratified` first divided the interval [20;40] into 32 slices (strata) of equal area for both `x` and `y`. Then it shuffled the order of these slices at random for both `x` and `y`. Then it picked a random number from within each slice (stratum).
- `RandomiserSobolSequence` picked the first 32 number from the Sobol' sequence for two variables and assigned them to `x` and `y`.

At the bottom you see the distribution of `x`. The Monte Carlo method yielded the least uniform and the Sobol' sequence the most uniform distribution.

At the top you see the joint distribution, where the Sobol' sequence stands out with the best coverage of the 2-dimensional variable space; this is what Sobol' designed his sequence for. Moreover, the good coverage of Sobol' numbers is still obtained with many variables in n-dimensional space.

The `randomiser` box is ignorant about the statistical distributions that are generated from. Behind the scenes it just produces random numbers in the interval [0;1], which are then grabbed by its sibling random boxes (in this example

RandomUniform) and transformed into the appropriate distribution. For another distribution than uniform, you can check out RandomNormal, for example.

Find more examples in RandomiserBase.

1.58 Class: RandomiserStratified

1.58.1 Inputs

Parameter	Type	Default	Description
`iteration`	`int`	a)	
`iterations`	`int`	b)	
`doSensitivityAnalysis`	`bool`	`FALSE`	Carry out a sensitivity analysis?
`bootstrapSize`	`int`	`1000`	Size of bootstrap sample (cheap in computation time); only used in sensitivity analysis
`seed`	`int`	`0`	Seed for random numbers; if this is zero a random seed value will be used
`drawAtInitialize`	`bool`	`FALSE`	Draw a value when a box is initialized?
`drawAtReset`	`bool`	`TRUE`	Draw a value when a box is reset?
`drawAtUpdate`	`bool`	`FALSE`	Draw a value when a box is updated?

a) `/.[iteration]`
b) `/.[iterations]`

1.58.2 Outputs

Parameter	Type	Default	Description
`numVariables`	`int`	`0 [0;inf)`	Number of variables which are randomised

See RandomiserSobolSequence.

1.59 Class: Ratio

1.59.1 Inputs

Parameter	Type	Default	Description
`denominator`	`double`	`0.0`	Number to be divided
`divisor`	`double`	`0.0`	The divisor
`zeroTolerance`	`double`	`0.0`	Used for infinity check
`allowInfinityResult`	`bool`	`TRUE`	Will yield an error if `divisor < zeroTolerance`

1.59.2 Outputs

Parameter	Type	Default	Description
`value`	`double`	`0.0`	The ratio

1.60 Class: Records

1.60.1 Inputs

Parameter	Type	Default	Description
`fileName`	`string`	a)	Name of file with records; columns separated by white space
`dateColumnName`	`string`	`"Date"`	Name of column with date
`timeColumnName`	`string`	`"Time"`	Name of column with time
`cycle`	`bool`	`FALSE`	Cycle back to start at end of file? Forces ignoreYear=true
`ignoreYear`	`bool`	`FALSE`	Ignore year when synchronising with the calendar
`calendarDateTime`	`datetime`	b)	

a) `"records.txt"`
b) `calendar[dateTime]`

1.60.2 Outputs

Parameter	Type	Default	Description
`firstDateTime`	`datetime`	`null`	Date-time stamp of the first line in the file
`lastDateTime`	`datetime`	`null`	Date-time stamp of the last line in the file
`currentDateTime`	`datetime`	`null`	Date-time stamp of the current outputs
`nextDateTime`	`datetime`	`null`	Date-time stamp of the next outputs
`currentDate`	`date`	`null`	Date stamp of the current outputs
`nextDate`	`date`	`null`	Date stamp of the next outputs
`currentTime`	`time`	`null`	Time stamp of the current outputs
`nextTime`	`time`	`null`	Time stamp of the next outputs

- Daily readings
- Interpolation
- Hourly readings
- Ignoring the year
- Extrapolation
- Recycling the year

1.60.3 Daily readings

You use a `Records` box to read logged data, i.e., data with a time stamp, typically a weather file. Here is one example showing the first and last lines of flakkebjerg 2005.txt ▾ :

```
      Date        Tmin Tmax  Tavg  I
01/01/2005  0    4.7   2.8   1.4
02/01/2005  0.7  6.2   3.6   1.3
03/01/2005  1.4  7     4.1   2
...
29/12/2005  -3.6 0.9  -2.2   1.3
30/12/2005  -2.4 1.3  -0.3   1.7
31/12/2005  -3  -0.8  -2.5   1.5
```

The input file must be a column-oriented text file with columns separated by whitespace (any number of blanks and tab stops between each column) and one row for each records. Columns must be complete; you cannot have an empty value. Rows, on the other hand, need not be complete. If some records are missing, say a few days due to a

malfunctioning weather station, then the `Records` box will automatically interpolate (linearly) between neighboring records.

The `Records` box must be accompanied by a `Calendar` box. It is the `Calendar` box that keeps track of time during a simulation, not the `Records` box. In this example, the simulation runs through three days in December:

```
// records1.box
Simulation sim {
  .steps = 3
  Calendar calendar {
    .begin = 15/12/2005
  }
  Records weather {
    .fileName = "../weather/flakkebjerg 2005.txt"
  }
  OutputR {
     OutputText {
       .ports = calendar[date] | weather[Tmin] | weather[Tmax]
     }
  }
}
```

We get this output from the simulation, as we would expect:

```
> load demo/records/records1.box
```

Each of the columns in the input file produces an output port with the same name as the column heading.

1.60.4 Interpolation

The temperatures are daily estimates, which for interpolation purposes are assumed to have occured at midnight. We can explore this by changing the time step to 8 hours:

```
// records2.box
Simulation sim {
  .steps = 6
  Calendar calendar {
    .begin = 15/12/2005
    .timeStep = 8
    .timeUnit = "h"
  }
  Records weather {
    .fileName = "../weather/flakkebjerg 2005.txt"
  }
  OutputR {
     OutputText {
       .ports = calendar[dateTime] | weather[Tmin] | weather[Tmax]
     }
  }
}
```

The output hits the same values as before every midnight (`head` shows only the first 6 rows of output, write `head 10` to see up to ten lines):

```
> head 10
```

Pitfalls. Plants and animals have a diurnal rythm. Be careful that your model makes sense, if you decrease the time step to less than 1 day. If use a daily accumulated variable like precipitation, you need to transform the data. If your

time step is 1 hour, for example, then you must divide the precipitation data in the input file by 24 before using it in a `Records` box.

1.60.5 Hourly readings

If you model runs on a finer time scale, you may have records at an hourly interval, or even down to seconds, corresponding to the smallest time step supported by `Calendar` (1 second). You may also have records at irregular intervals with a time stamp of date and time. In any case, you provide the time of the day in a separate column, as exemplified by the file copenhagen-may-hourly.txt ▾ , which contains hourly data for the first half of May:

```
Date       Time  Tair Rhair GlobRad DifRad Windspeed
01-05-2001 00:00 4.7  85     0      0      2.6
01-05-2001 01:00 4.1  84     0      0      2.6
01-05-2001 02:00 3.8  84     0      0      2.6
...
15-05-2001 22:00 7.4  83     0      0      5.8
15-05-2001 23:00 6.7  84     0      0      5.5
16-05-2001 00:00 6.1  85     0      0      5.1
```

In our boxscript, we specify the period of the simulation in the `Calendar` box:

```
// records3.box
Simulation sim {
  Calendar calendar {
    .begin = 01/05/2001T00:00
    .end   = 06/05/2001T24:00
    .timeStep = 15
    .timeUnit = "m"
  }
  Records weather {
    .fileName = "../weather/copenhagen-may-hourly.txt"
  }
  OutputR {
    PageR {
      .xAxis = calendar[dateTime]
      PlotR {
       .ports = weather[Tair] | weather[GlobRad] |weather[Windspeed]
      }
    }
  }
}
```

We let the simulation run with a 15-minute time step, which is a finer time resolution than given in the weather file. The `Records` box will oblige by interpolating the missing data, as you can see here:

```
> ht 3
```

The plot looks like this:

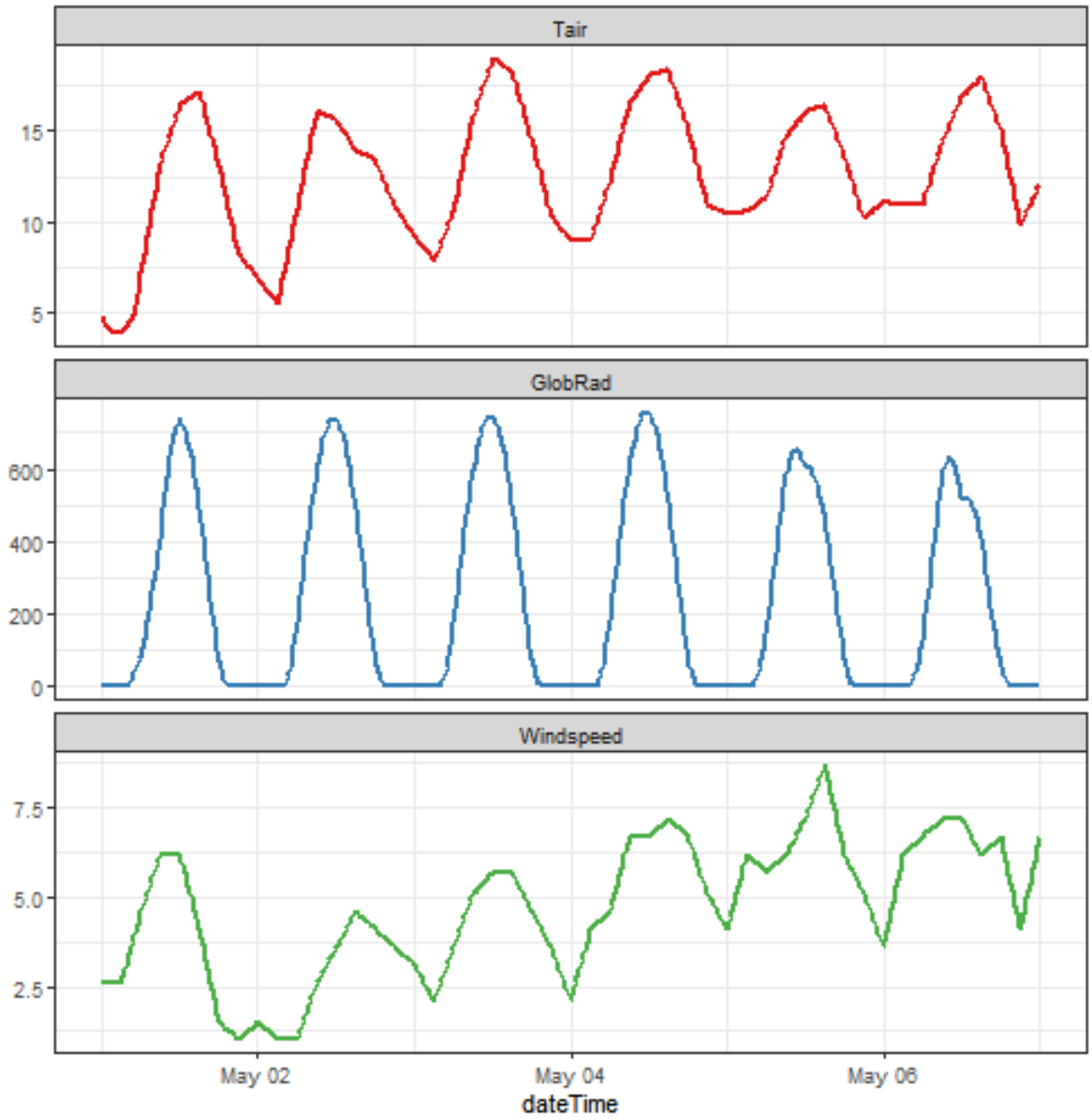


Figure 9: Records example

The two flavours of midnight. The time stamps 2001/05/06T24:00:00 and 2001/05/07T00:00:00 are equivalent because 24 hours is translated into 0 hours (midnight) on the following day.

1.60.6 Ignoring the year

It happens quite often that you want to use a weather file without regards to the year noted in the time stamp. This is the purpose of `ignoreYear`, which defaults to `FALSE` but here set it to `TRUE`:

```
// records4.box
Simulation sim {
  .steps = 3
  Calendar calendar {
    .begin = 15/12/2022
  }
  Records weather {
    .fileName = "../weather/flakkebjerg 2005.txt"
    .ignoreYear = TRUE
  }
```

```
  ...
}
```

The Calendar box sets the simulation to begin on 15 December 2022. In response, the Records box will begin producing output from the first row that has a time stamp of 15 December. In this case, it wll be ignoring the time stamp's year (2005). As a result we get this output:

We do get the same output as previously, except the year now follows the `Calendar` box:

```
      date Tmin Tmax iteration step
2022/12/15  2.7  8.1         1    0
2022/12/16  1.3  4.5         1    1
2022/12/17 -4.2  0.7         1    2
2022/12/18 -2.8  1.6         1    3
```

1.60.7 Extrapolation

What happens if we run past the end of the weather file? Consider this:

```
// records6.box
Simulation sim {
  .steps = 5
  Calendar calendar {
    .begin = 28/12/2005
  }
  Records weather {
    .fileName = "../weather/flakkebjerg 2005.txt"
  }
  ...
}
```

The `Records` box responds by extrapolating the final reading:

```
      date Tmin Tmax iteration step
2005/12/28 -3.2 -1.4         1    0
2005/12/29 -3.6 -0.9         1    1
2005/12/30 -2.4  1.3         1    2
2005/12/31 -3.0 -0.8         1    3
2006/01/01 -3.0 -0.8         1    4
2006/01/02 -3.0 -0.8         1    5
```

1.60.8 Recycling the year

If your weather file goes from 1 January to 31 December, or if it otherwise forms a whole year from beginning to end, e.g. from 21 March one year to 20 March the following year, then you can ask the Records box to recycle the readings. In that way, you can use one year's weather data to simulate several years. All right, that's a quick and dirty way of doing it, but sometimes we just want it like that.

You achieve this behaviour by setting the `cycle` flag `TRUE` (it defaults to `FALSE`). If `cycle` is `TRUE` then `ignoreYear` will always be forced to `TRUE` as well, since the year registered in the input file has become irrelevant. Let's try it:

```
// records7.box
Simulation sim {
  .steps = 5
  Calendar calendar {
    .begin = 30/12/2022
  }
  Records weather {
    .fileName = "../weather/flakkebjerg 2005.txt"
    .cycle = TRUE
```

```
  }
  ...
}
```

We have set the calendar to reach past the end of the year. The output shows that we have successfully achieved to wrap around the weather file on 31 December and cycle back to its beginning on 1 January:

```
      date Tmin Tmax iteration step
2022/12/30 -2.4  1.3         1    0
2022/12/31 -3.0 -0.8         1    1
2023/01/01  0.0  4.7         1    2
2023/01/02  0.7  6.2         1    3
2023/01/03  1.4  7.0         1    4
2023/01/04  2.6  7.7         1    5
```

1.61 Class: RunningAverageSteps

1.61.1 Inputs

Parameter	Type	Default	Description
`initial`	`double`	`0.0`	Value at reset
`input`	`double`	`0.0`	The input to sum or average
`count`	`int`	`1 >0`	Number of steps to average over

1.61.2 Outputs

Parameter	Type	Default	Description
`value`	`double`	`0.0`	Running sum or average

1.61.3 Usage

`RunningAverageSteps` computes running average by no. of steps.

1.62 Class: RunningAverageTimed

1.62.1 Inputs

Parameter	Type	Default	Description
`initial`	`double`	`0.0`	Value at reset
`input`	`double`	`0.0`	The input to sum or average
`timeWindow`	`double`	`1.0 int>0`	Time window to average over in units of timeUnit
`timeUnit`	`string`	`"d" y|d|h|m|s`	Unit of time window
`timeStepSecs`	`double`	a)	

a) `calendar[timeStepSecs]`

1.62.2 Outputs

Parameter	Type	Default	Description
`value`	`double`	`0.0`	Running sum or average
`count`	`int`	`0 integer`	Number of steps averages over

1.62.3 Usage

`RunningAverageTimed` computes running average in a time window.

1.63 Class: RunningSumSteps

1.63.1 Inputs

Parameter	Type	Default	Description
initial	double	0.0	Value at reset
input	double	0.0	The input to sum or average
count	int	1 >0	Number of steps to average over

1.63.2 Outputs

Parameter	Type	Default	Description
value	double	0.0	Running sum or average

1.63.3 Usage

RunningSumSteps computes running sum by no. of steps.

1.64 Class: RunningSumTimed

1.64.1 Inputs

Parameter	Type	Default	Description
initial	double	0.0	Value at reset
input	double	0.0	The input to sum or average
timeWindow	double	1.0 int>0	Time window to average over in units of timeUnit
timeUnit	string	"d" y|d|h|m|s	Unit of time window
timeStepSecs	double	a)	

a) calendar[timeStepSecs]

1.64.2 Outputs

Parameter	Type	Default	Description
value	double	0.0	Running sum or average
count	int	0 integer	Number of steps averages over

1.64.3 Usage

RunningSumTimed computes running sum in a time window.

1.65 Class: Sah

1.65.1 Inputs

Parameter	Type	Default	Description
temperature	double	0.0 °C	Temperature
value	double	0.0 kg/m3	Saturated absolute humidity

1.65.2 Usage

Sah computes saturated absolute humidity.

1.66 Class: Sandy

1.66.1 Inputs

Parameter	Type	Default	Description
xMin	double	0.0	
xMax	double	1.0	
yMax	double	1.0	
a	double	1.0	
b	double	1.0	
x	double	0.0	

1.66.2 Outputs

Parameter	Type	Default	Description
value	double	0.0	

1.66.3 Usage

`Sandy` the Sandy equation (experimental).

1.67 Class: Scenarios

1.67.1 Inputs

Parameter	Type	Default	Description
fileName	string	a)	Text file with columns, one for each output produced. One iteration is run for each line
title	string	"Scenario"	Name of column that holds title of scenario (optional)

a) `"scenarios.txt"`

1.67.2 Outputs

Parameter	Type	Default	Description
iterations	int	0	Number of iterations needed to run scenarios

1.67.3 Usage

`Scenarios` runs scenarios from a text file.

1.68 Class: SelectFile

1.68.1 Inputs

Parameter	Type	Default	Description
folder	string		Relative path to folder
filter	string	"*.txt"	File filter
selectFileNumber	int	1	Which file to pick
fileNumberOffset	int	1	Offset for fileNumber, typically 0 or 1
showFileNames	bool	TRUE	Show set of file names?

1.68.2 Outputs

Parameter	Type	Default	Description
`numFiles`	`int`	`0`	Number of matching files
`fileName`	`string`		Name of selected file
`fileNamePath`	`string`		Name of selected file with absolute path

1.68.3 Usage

`SelectFile` selects a file from a folder.

1.69 Class: SequenceByIteration

1.69.1 Inputs

Parameter	Type	Default	Description
`counter`	`int`	`0`	Current counter into values
`min`	`double`	`0.0`	Minimum value in sequence
`max`	`double`	`1.0`	Maximum value in sequence
`by`	`double`	`0.1`	Maximum value in sequence

1.69.2 Outputs

Parameter	Type	Default	Description
`numValues`	`int`	`0`	Number of values
`values`	`vec_double`	`null`	Vector of all values
`value`	`double`	`0.0`	Current value
`iterations`	`int`	`0`	The number of iterations needed to run through all values

1.69.3 Usage

`SequenceByIteration` produces a sequence of numbers changing by every iteration.

1.70 Class: SequenceByStep

1.70.1 Inputs

Parameter	Type	Default	Description
`counter`	`int`	`0`	Current counter into values
`min`	`double`	`0.0`	Minimum value in sequence
`max`	`double`	`1.0`	Maximum value in sequence
`by`	`double`	`0.1`	Maximum value in sequence

1.70.2 Outputs

Parameter	Type	Default	Description
`numValues`	`int`	`0`	Number of values
`values`	`vec_double`	`null`	Vector of all values
`value`	`double`	`0.0`	Current value
`steps`	`int`	`0`	The number of steps needed to run through all values

1.70.3 Usage

`SequenceByStep` produces a sequence of numbers changing by every step.

1.71 Class: SeriesByIteration

1.71.1 Inputs

Parameter	Type	Default	Description
`counter`	`int`	`0`	Current counter into values
`numbers`	`vec_double`	`null`	Vector holding a series of values

1.71.2 Outputs

Parameter	Type	Default	Description
`numValues`	`int`	`0`	Number of values
`values`	`vec_double`	`null`	Vector of all values
`value`	`double`	`0.0`	Current value
`iterations`	`int`	`0`	The number of iterations needed to run through all values

1.71.3 Usage

`SeriesByIteration` produces a given series of numbers changing by every iteration.

1.72 Class: SeriesByStep

1.72.1 Inputs

Parameter	Type	Default	Description
`counter`	`int`	`0`	Current counter into values
`numbers`	`vec_double`	`null`	Vector holding a series of values

1.72.2 Outputs

Parameter	Type	Default	Description
`numValues`	`int`	`0`	Number of values
`values`	`vec_double`	`null`	Vector of all values
`value`	`double`	`0.0`	Current value
`steps`	`int`	`0`	The number of steps needed to run through all values

1.72.3 Usage

`SeriesByStep` produces a series of numbers changing by every step.

1.73 Class: Simulation

1.73.1 Inputs

Parameter	Type	Default	Description
`iterations`	`int`	`1`	Number of iterations to run
`steps`	`int`	a)	
`stopIteration`	`bool`	`FALSE`	Stop running after this iteration?
`stopStep`	`bool`	`FALSE`	Stop running after this step?

Parameter	Type	Default	Description
silent	bool	FALSE	Run without scrolling progress messages?
unattended	bool	FALSE	An unattended run will finish without accessing clipboard

a) `if exists(Calendar::*[steps]) then Calendar::*[steps] else 1`

1.73.2 Outputs

Parameter	Type	Default	Description
iteration	int	0	Iteration number (1,2,...)
step	int	0	Step number (0,1,2,...)
finalStep	int	0	Last completed step in simulation
executionTime	int	0	Duration of simulation run (ms)
hasRun	bool	FALSE	Has this simulation been run?
hasError	bool	FALSE	Did an error occur during simulation run?
errorMsg	string		Error message

1.73.3 Usage

`Simulation` runs a simulation.

1.74 Class: SineWaveTemperature

1.74.1 Inputs

Parameter	Type	Default	Description
recordsName	string	"weather"	Name of Records box holding temperature
TminName	string	"Tmin"	Name of column holding daily min. temperature
TmaxName	string	"Tmax"	Name of column holding daily max. temperature
morningOffset	double	0.5 h	Hours after sunrise when temperature is at minimum
afternoonOffset	double	3.0 h	Hours after noon when temperature is at maximum
time	time	a)	
sunrise	time	b)	
solarTime	time	c)	
timeStep	int	d)	
timeUnit	string	e)	

a) `calendar[time]`
b) `calendar[sunrise]`
c) `calendar[trueSolarTime]`
d) `calendar[timeStep]`
e) `calendar[timeUnit]`

1.74.2 Outputs

Parameter	Type	Default	Description
TminYesterday	double	0.0	Yesterday's Tmin
TmaxYesterday	double	0.0	Yesterday's Tmax

Parameter	Type	Default	Description
TminToday	double	0.0	Today's Tmin
TmaxToday	double	0.0	Today's Tmax
TminTomorrow	double	0.0	Tomorrow's Tmin
TmaxTomorrow	double	0.0	Tomorrow's Tmax
value	double	0.0	Sine-interpolated temperature

1.74.3 Usage

SineWaveTemperature computes sine-wave interpolation from daily min-max temperatures.

1.75 Class: SobolGFunction

1.75.1 Inputs

Parameter	Type	Default	Description
a	vec_double	null	Vector of parameters
x	vec_double	null [0;1]	Vector of inputs

1.75.2 Outputs

Parameter	Type	Default	Description
value	double	0.0	B value

1.75.3 Usage

SobolGFunction computes Sobol' g function.

1.76 Class: Split

1.76.1 Inputs

Parameter	Type	Default	Description
whole	double	0.0	The number to be split
proportions	vec_double	null	Vector of proportions; sum<=1

1.76.2 Outputs

Parameter	Type	Default	Description
pieces	vec_double	null	Vector holding proportions of whole; same length as proportions

1.76.3 Usage

Split splits a number into proportional pieces (max. 5).

1.77 Class: Stage

1.77.1 Inputs

Parameter	Type	Default	Description
initial	double	0.0	Initial amount of inflow
k	int	30	Distribution parameter; use small k for larger dispersion in output
duration	double	100.0	Average delay between inflow and outflow

Parameter	Type	Default	Description
timeStep	double	1.0	Time step
growthFactor	double	1.0	Factor by which outflow will be scaled relative to inflow
instantLossRate	double	0.0	Mortality applied before inflow is added [0..1]
phaseInflow	vec_double	null	Phase inflow cohorts (vector)
zeroLimit	double	1e-12	Cut contents to zero when total is less than this
inflow	double	0.0	Amount of inflow
phaseOutflowProportion	double	0.0	Proportion that will change phase in next time step

1.77.2 Outputs

Parameter	Type	Default	Description
content	double	0.0	Total content inside the stage
value	double	0.0	Synonym for 'content'
inflowSum	double	0.0	Sum of current stage inflow (scalar)
inflowTotal	double	0.0	Accumulated total stage inflow (scalar)
phaseInflowSum	double	0.0	Sum of current phase inflow cohorts (scalar)
phaseInflowTotal	double	0.0	Accumulated total phase inflow (scalar)
outflowSum	double	0.0	Sum of current stage outflow (scalar)
outflowTotal	double	0.0	Accumulated total stage outflow (scalar)
phaseOutflowSum	double	0.0	Sum of current phase outflow cohorts (scalar)
phaseOutflowTotal	double	0.0	Accumulated total phase outflow (scalar)
phaseOutflow	vec_double	null	Phase outflow cohorts (vector)
growth	double	0.0	Change in content+outflow during this time step
outflow	double	0.0	Outflow emerging from the stage

1.77.3 Usage

`Stage` models a distributed delay.

1.78 Class: StageAndPhase

1.78.1 Inputs

Parameter	Type	Default	Description
initial	double	0.0	Initial amount of inflow
k	int	30	Distribution parameter; use small k for larger dispersion in output
duration	double	100.0	Average delay between inflow and outflow
timeStep	double	1.0	Time step
growthFactor	double	1.0	Factor by which outflow will be scaled relative to inflow
instantLossRate	double	0.0	Mortality applied before inflow is added [0..1]
phaseInflow	vec_double	null	Phase inflow cohorts (vector)
zeroLimit	double	1e-12	Cut contents to zero when total is less than this

Parameter	Type	Default	Description
inflow	vec_double	null	Amount of inflow (vector)
scalarInflow	double	0.0	Amount of inflow (scalar)
phaseK	int	30	Phase distribution parameter
phaseDuration	double	100.0	Average delay between inflow and outflow through phase
phaseTimeStep	double	1.0	Time step for phase development

1.78.2 Outputs

Parameter	Type	Default	Description
content	double	0.0	Total content inside the stage
value	double	0.0	Synonym for 'content'
inflowSum	double	0.0	Sum of current stage inflow (scalar)
inflowTotal	double	0.0	Accumulated total stage inflow (scalar)
phaseInflowSum	double	0.0	Sum of current phase inflow cohorts (scalar)
phaseInflowTotal	double	0.0	Accumulated total phase inflow (scalar)
outflowSum	double	0.0	Sum of current stage outflow (scalar)
outflowTotal	double	0.0	Accumulated total stage outflow (scalar)
phaseOutflowSum	double	0.0	Sum of current phase outflow cohorts (scalar)
phaseOutflowTotal	double	0.0	Accumulated total phase outflow (scalar)
phaseOutflow	vec_double	null	Phase outflow cohorts (vector)
growth	double	0.0	Change in content+outflow during this time step
outflow	vec_double	null	Stage outflow cohorts (vector)

1.78.3 Usage

StageAndPhase models a 2-D distributed delay.

1.79 Class: Sum

1.79.1 Inputs

Parameter	Type	Default	Description
values	vec_double	null	Input values

1.79.2 Outputs

Parameter	Type	Default	Description
value	double	0.0	Sum of inputs; zero if no inputs
count	int	0	Number of inputs

1.79.3 Usage

Sum computes sum of inputs.

1.80 Class: Sun

1.80.1 Inputs

Parameter	Type	Default	Description
latitude	double	a)	
longitude	double	b)	
dateTime	datetime	c)	
timeZone	double	d)	
altitude	double	0.0 m	Altitude above sea level

a) calendar[latitude]
b) calendar[longitude]
c) calendar[dateTime]
d) calendar[timeZone]

1.80.2 Outputs

Parameter	Type	Default	Description
sunrise	time	null time	Time of sunrise
noon	time	null time	Time of nonw
sunset	time	null time	Time of sunset
solarTime	time	null time	Solar time (noon=12:00:00
dayLength	double	0.0 h	Day length from sunrise to sunset
elevation	double	0.0 [-90;90]	Solar angle to horizon
azimuth	double	0.0 [0;360]	Compass direction to sun (north=0)
isDay	bool	FALSE	Is the sun up?
isNight	bool	FALSE	Has the sun set?
solarConstant	double	0.0 W/m2	The irradiation at the top of the atmosphere
angot	double	0.0 W/m2	The irradiation on Earth surface under optimal atmospheric conditions

1.80.3 Usage

Sun computes sun-earth astronomy.

1.81 Class: SunDiffuseRadiation

1.81.1 Inputs

Parameter	Type	Default	Description
globalRadiation	double	0.0 W/2	Global radiation
angotRadiation	double	sun[angot]	

1.81.2 Outputs

Parameter	Type	Default	Description
value	double	0.0 [0;1]	Proportion diffuse light

1.81.3 Usage

SunDiffuseRadiation computes proportion diffuse global radiation.

1.82 Class: SuperFunctionalResponse

1.82.1 Inputs

Parameter	Type	Default	Description
attacker	double	0.0 #attackers	Attacker density
host	double	0.0 #hosts	Resource density
demand	double	0.0 #attacks	Number of attacks demanded
attackRate	double	0.0	#attacks per #host per #attacker per time unit
timeStep	double	1.0 Time unit	Time step

1.82.2 Outputs

Parameter	Type	Default	Description
supply	double	0.0 #attacks	Number of attacks accomplished
hostsAttacked	double	0.0 #hosts	The number of hosts attacked
propHostsAttacked	double	0.0 [0;1]	Proportion of hosts attacked
propAttackersAttacked	double	0.0 [0;1]	Proportion of successful attackers
sdRatio	double	0.0 [0;1]	Attackers's supply/demand ratio

1.82.3 Usage

SuperFunctionalResponse computes a Frazer-Gilbert functional response.

1.83 Class: SupplyBudget

1.83.1 Inputs

Parameter	Type	Default	Description
supplyGross	double	0.0 #prey	Gross supply
demandNet	double	0.0 #prey	Net demand
demandResp	double	0.0 #prey	Respiration demand
egested	double	0.0 [0;1]	Egested or unused fraction of prey
conversionCost	double	0.0 [0;1]	Cost of converting prey into consumer)

1.83.2 Outputs

Parameter	Type	Default	Description
supplyNet	double	0.0 #prey	Net supply for predator assimilation
egestedAmount	double	0.0 #prey	Egested (or unused) amount of resource
conversionCostAmount	double	0.0 #prey	Cost of converting resource into consumer
sdRatioNet	double	0.0 [0;1]	Predator's net supply/demand ratio

1.83.3 Usage

SupplyBudget computes net supply from energy budget.

1.84 Class: ThresholdSignal

1.84.1 Inputs

Parameter	Type	Default	Description
`initialSignal`	`double`	`0.0`	Value of `signal` at reset
`initialFlag`	`bool`	`FALSE`	Value of flag at reset
`input`	`double`	`0.0`	Input that is compared to threshold
`threshold`	`double`	`0.0`	Threshold value
`flagAbove`	`bool`	`TRUE`	Raise flag when input is at or above threshold?
`signalFlagged`	`double`	`1.0`	Signal when flagged
`signalUnflagged`	`double`	`0.0`	Signal when not flagged

1.84.2 Outputs

Parameter	Type	Default	Description
`signal`	`double`	`0.0`	Value of the signal
`flagIsUp`	`bool`	`FALSE`	Is the `signal` on?
`flagJustRaised`	`bool`	`FALSE`	Was the `signal` just set on?
`flagJustLowered`	`bool`	`FALSE`	Was the `signal` just set off?
`value`	`double`	`0.0`	Synonym for `signal`

1.84.3 Usage

`ThresholdSignal` boxes::ThresholdSignal sets a signal according to a threshold value.

1.85 Class: TimeToHours

1.85.1 Inputs

Parameter	Type	Default	Description
`time`	`time`	`00:00:00` `time`	Time of day

1.85.2 Outputs

Parameter	Type	Default	Description
`value`	`double`	`0.0`	Hours since midnight

1.85.3 Usage

`TimeToHours` converts time to hours since midnight.

1.86 Class: VectorScaled

1.86.1 Inputs

Parameter	Type	Default	Description
`vector`	`vec_double`	`null`	Vector to be scaled
`factor`	`double`	`0.0`	Scaling factor

1.86.2 Outputs

Parameter	Type	Default	Description
`value`	`vec_double`	`null`	Scaled vector

1.86.3 Usage

`VectorScaled` scales a vector by a factor.

1.87 Class: VectorSum

1.87.1 Inputs

Parameter	Type	Default	Description
`input`	`string`		Path to vectors (must have same length)

1.87.2 Outputs

Parameter	Type	Default	Description
`value`	`vec_double`	`null`	Sum; a vector of same length as the inputs

1.87.3 Usage

`VectorSum` computes the vector sum of many vectors.

1.88 Class: boxesDocumentation

1.88.1 Inputs

Parameter	Type	Default	Description
`title`	`string`	a)	
`authorNames`	`vec_string`	`null`	
`authorInstitutes`	`vec_string`	`null`	
`authorEmails`	`vec_string`	`null`	

a) `Basic building blocks`

1.88.2 Usage

`boxesDocumentation` base::Documentation provides documentation.

2 Module: callosobruchus

2.1 Class: DensityDependent

2.1.1 Inputs

Parameter	Type	Default	Description
seedCapacity	int	9	Number of beetles one seed can sustain
beetlesProducedTotal	double	0.0	Total number of beetles produced since stocking
seedsTotal	double	0.0	Total number of seeds stocked
k	double	8.17	Dispersion of the negative binomial
exponent	double	-0.496	Negative exponential response

2.1.2 Outputs

Parameter	Type	Default	Description
avgBeetlesProducedPerSeed	double	0.0	Average number of eggs per seed, laid since stocking
survival	double	0.0 [0;1]	Survival through juvenile life stages
damage	double	0.0 [0;1]	Proportion of seeds damaged

2.1.3 Usage

DensityDependent models density-dependt survival and damage.

2.2 Class: Fecundity

2.2.1 Inputs

Parameter	Type	Default	Description
sexRatio	double	0.5	Proportion of females
temperature	double	0.0 °C	Store temperature
slope	double	1.279 eggs per oC	Fecundity response on temperature
intercept	double	-23.1 eggs	Fecundity response on temperature
minTemperature	double	25.0 °C	No extrapolation below this temperature
maxTemperature	double	40.0 °C	No extrapolation above this temperature

2.2.2 Outputs

Parameter	Type	Default	Description
value	double	0.0 eggs	Total eggs laid per female

2.2.3 Usage

Fecundity models female life-time fecundity.

2.3 Class: Stock

2.3.1 Inputs

Parameter	Type	Default	Description
size	double	100.0 kg	Weight of seeds stored

Parameter	Type	Default	Description
`seedsPerKg`	`double`	`7000.0 per kg`	Number of seeds per kg put on storage

2.3.2 Outputs

Parameter	Type	Default	Description
`seedsTotal`	`double`	`0.0`	Total no. of seeds

2.3.3 Usage

`Stock` models female life-time fecundiy.

2.4 Class: callosobruchusDocumentation

2.4.1 Inputs

Parameter	Type	Default	Description
`title`	`string`	a)	
`authorNames`	`vec_string`	`null`	
`authorInstitutes`	`vec_string`	`null`	
`authorEmails`	`vec_string`	`null`	

a) `callosobruchus model`

2.4.2 Usage

`callosobruchusDocumentation` base::Documentation provides documentation.

3 Module: cmbp

3.1 Class: DevRate

3.1.1 Inputs

Parameter	Type	Default	Description
b1	double	0.0	Regression parameter
b2	double	0.0	Regression parameter
b3	double	0.0	Regression parameter
b4	double	0.0	Regression parameter
T	double	0.0	Ambient temperature (oC)

3.1.2 Outputs

Parameter	Type	Default	Description
value	double	0.0	Daily development rate

3.1.3 Usage

`DevRate` is a container of boxes and ports.

3.2 Class: Emigration

3.2.1 Inputs

Parameter	Type	Default	Description
e2	double	0.0	Regression parameter
e3	double	0.0	Regression parameter
N	double	0.0	Adult density

3.2.2 Outputs

Parameter	Type	Default	Description
value	double	0.0	Peak curve

3.2.3 Usage

`Emigration` is a container of boxes and ports.

3.3 Class: ExponentialPeak

3.3.1 Inputs

Parameter	Type	Default	Description
d1	double	0.0	Regression parameter
d2	double	0.0	Regression parameter
x	double	0.0	Independent variable
optimum	double	0.0	x-value at peak y

3.3.2 Outputs

Parameter	Type	Default	Description
value	double	0.0	Peak curve

3.3.3 Usage

`ExponentialPeak` is a container of boxes and ports.

3.4 Class: Fecundity

3.4.1 Inputs

Parameter	Type	Default	Description
`maxFecundity`	`double`	`700.0`	Life-time fecundity of a female at 30oC
`r`	`double`	`0.2359`	Exponential increase in fecundity with temperature
`T`	`double`	`0.0`	Store temperature (oC)
`sexRatio`	`double`	`0.5`	Sex ratio [0;1]

3.4.2 Outputs

Parameter	Type	Default	Description
`value`	`double`	`0.0`	Life-time fecundity scaled by temperature

3.4.3 Usage

`Fecundity` is a container of boxes and ports.

3.5 Class: GrainMoistureContent

3.5.1 Inputs

Parameter	Type	Default	Description
`c1`	`double`	`0.523`	Regression parameter
`c2`	`double`	`-0.00967`	Regression parameter
`c3`	`double`	`7.06e-05`	Regression parameter
`rh`	`double`	`0.0`	Ambient relative humidity (%)

3.5.2 Outputs

Parameter	Type	Default	Description
`value`	`double`	`0.0`	Grain moisture content (%)

3.5.3 Usage

`GrainMoistureContent` is a container of boxes and ports.

3.6 Class: GrainStore

3.6.1 Inputs

Parameter	Type	Default	Description
`Tmin`	`double`	`0.0`	Outdoors daily minimum temperature (oC)
`Tmax`	`double`	`0.0`	Outdoors daily maximum temperature (oC)
`Toffset`	`double`	`0.0`	Offset for temperature inside the store (oC)
`mass`	`double`	`500.0`	Mass of stored grain (kg)

3.6.2 Outputs

Parameter	Type	Default	Description
T	double	0.0	Store temperature (oC

3.6.3 Usage

GrainStore is a container of boxes and ports.

3.7 Class: Immigration

3.7.1 Inputs

Parameter	Type	Default	Description
number	double	0.0	Nuber of P.t. arriving
storeMass	double	500.0	Size of store (kg maize)
day	int	0	Day of immigration (no. of days after start)

3.7.2 Outputs

Parameter	Type	Default	Description
value	double	0.0	Immigrant increase in P.t. density (per kg

3.7.3 Usage

Immigration is a container of boxes and ports.

3.8 Class: SurvivalByDensity

3.8.1 Inputs

Parameter	Type	Default	Description
e1	double	0.0	Regression parameter
N	double	0.0	Adult density

3.8.2 Outputs

Parameter	Type	Default	Description
value	double	0.0	Peak curve

3.8.3 Usage

SurvivalByDensity is a container of boxes and ports.

4 Module: command

4.1 Class: commandDocumentation

4.1.1 Inputs

Parameter	Type	Default	Description
`title`	`string`	a)	
`authorNames`	`vec_string`	`null`	
`authorInstitutes`	`vec_string`	`null`	
`authorEmails`	`vec_string`	`null`	

a) `Commands for the prompt`

4.1.2 Usage

`commandDocumentation` base::Documentation provides documentation.